



Σavante

**Modelado de datos,
metodologías y reglas.
Entidades, atributos y
relaciones. Diseño de bases
de datos. Diseño lógico y físico.
El modelo lógico relacional.
Normalización**

ÍNDICE

1. Diseño de Bases de Datos	5
1.1. Introducción. Lógica de programación	6
1.2. Recordando conceptos	7
1.2.1. Procedimiento	8
1.2.2. Sistema Gestor de Base de Datos SGBD	8
1.2.2.1. Tipos de bases de datos por clasificaciones	9
1.2.2.1.1. Clasificación por modelo de datos	9
1.2.2.1.2. Clasificación por capacidades del sistema	11
1.2.2.1.3. Clasificación por propósito	11
1.2.2.1.4. Bases de datos especializadas	12
1.3. Objetivos y cualidades de un buen diseño	12
1.4. Fases del diseño de base de datos. Esquematización	14
2. Fase 1. Diseño conceptual	16
2.1. Esquema conceptual	17
2.1.1. El modelo conceptual	17
2.1.2. Modelo entidad-relación (E/R) de Peter Chen (1976)	18
2.1.2.1. Entidad (entity). Tipos	19
2.1.2.2. Atributo (attribute)	21
2.1.2.3. Relación entre entidades	24
2.1.2.3.1. Clasificación de las Relaciones en el Modelo E/R	26
2.1.2.4. Relaciones modelo extendido	26
2.1.2.4.1. Nombre	27
2.1.2.4.2. Cardinalidad	27
2.1.2.4.3. Tipo de Correspondencia	27
2.1.2.5. Jerarquías de generalización/especialización	28
2.2. Construcción del modelo conceptual de datos. Metodología	29
2.3. Diagramas de flujo de datos	34
2.3.1. Elementos. Notación	35
2.3.2. Representación Gráfica de la Notación	38
2.3.3. Descomposición o explosión en niveles	39

3. Fase 2. Diseño lógico	44
3.1. Flujogramas	45
3.1.1. Tipos	45
3.1.2. Elementos y su notación	46
3.1.3. Reglas de construcción	49
3.1.4. Ejemplos de Flujogramas	50
3.1.5. Pensar como un programador	51
3.2. Información de la carga y criterios de rendimiento	53
3.3. El modelo lógico relacional (B.D. Relacional)	54
3.3.1. Conceptos básicos	55
3.3.2. Reglas de Integridad en el Modelo Relacional	57
3.3.2.1. Regla de integridad de entidad de la clave primaria	57
3.3.2.2. Regla de integridad de unicidad de la clave primaria	58
3.3.2.3. Regla de integridad referencial	58
3.3.2.4. Regla de Datos-Atributos requeridos	61
3.3.2.5. Regla de integridad de dominio	61
3.3.2.6. Reglas de negocio	61
3.3.3. Metodología de diseño lógico en el modelo relacional	61
3.4. Normalización	68
3.4.1. Formas normales	69
3.4.1.1. 1Fn	70
3.4.1.2. 2Fn	72
3.4.1.3. 3Fn	73
3.4.1.4. FNBC (forma normal de Boyce-Codd)	74
3.4.1.5. 4Fn	76
3.4.1.6. 5Fn	78
3.4.2. Desnormalización	81
3.5. Partición de relaciones	82
3.6. Optimización	83

4. Fase 3. Diseño físico	84
4.1. Fases de diseño físico	85
4.1.1. Traducción del esquema lógico global al SGBD específico	86
4.1.2. Diseño de la representación física	87
4.1.2.1. Pasos para la representación física	87
4.1.3. Diseño de los mecanismos de seguridad	88
4.1.4. Monitorización y optimización del sistema	88
5. Bibliografía	89



1. Diseño de Bases de Datos



Fuente: Public domain vectors

El diseño de la base de datos es la base de todo el funcionamiento de un programa, nos debe proporcionar las mejores condiciones del tratamiento de datos para que el programa que creemos sea eficaz y eficiente.

Todo debe estar totalmente especificado y concretado al máximo.

El diseño de bases de datos, lo realiza un analista y/o un programador.



Recuerda ver las clases emitidas en Temario Audiovisual

Las clases impartidas en directo y disponibles en Campus, en Temario Audiovisual te ayudarán al entendimiento de la unidad, y además pueden tener información adicional.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)



1.1. Introducción. Lógica de programación

Lo fundamental es que conozcas bien las necesidades del usuario para que puedas diseñar correctamente la base de datos y crear el programa.

Debes entender bien lo que el usuario pide y necesita, y adelantarte añadiendo las necesidades que podrá tener posteriormente.

Veamos cómo debe pensar un programador para poder realizar el diseño de una Base de Datos.

Si tenemos un concesionario de Venta de coches, turismos de segunda mano, qué datos hay que conocer y tener almacenados.

- **Coches:** matrícula, marca, modelo, motor, nº de puertas, color, ...
- **Vendedores:** código de vendedor, nombre y apellido, DNI, teléfono...
- **Clientes:** código de cliente, nombre y apellido, DNI, teléfono...

Estos datos los reflejamos en tablas, que tienen filas y columnas:

TABLA COCHES						
Nombre Dato	Matricula	Marca	Modelo	Motor	Nº de puertas	Color
Fila 1	Z-1010-B	Seat	León	Diesel	5	Verde
Fila 2	M-2189-X	Ford	Focus	Gasolina	3	Rojo
Fila 3	M-8596-J	Citroën	Xara	Diesel	5	Verde
Columna 1	Columna 2	Columna 3	Columna 4	Columna 5	Columna 6	Columna 7

A cada fila de información excepto la del nombre del dato, le corresponde un único coche.

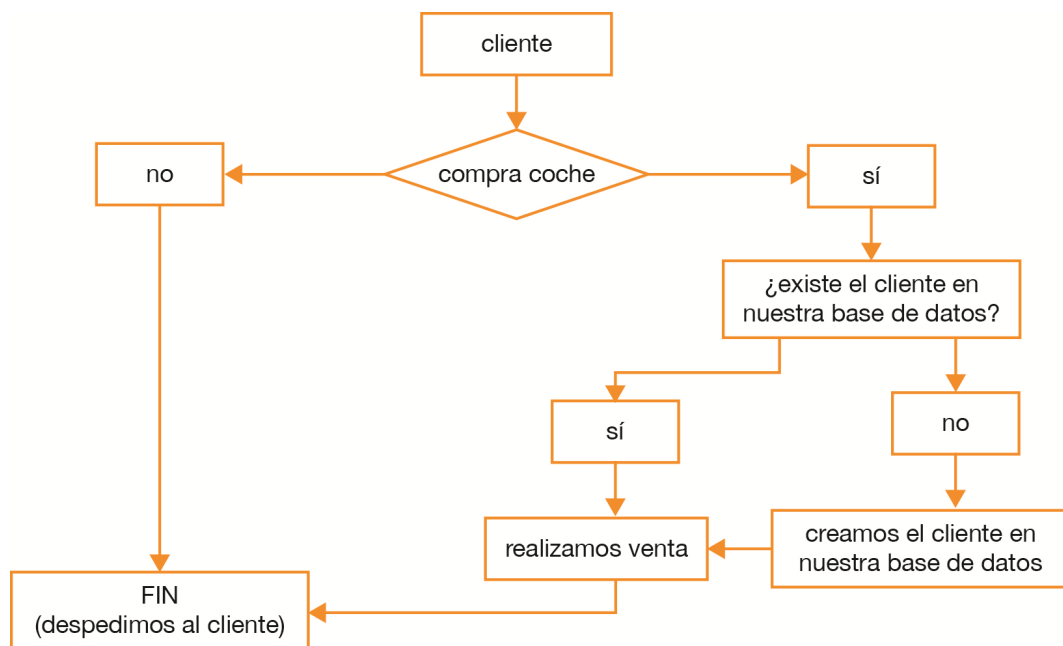
A cada columna de información se le asignará un valor que podrá estar o no repetido, (color verde aparece 2 veces) excepto la columna Matricula que será única (no puede haber 2 coches con la misma matricula). Por tanto, nuestra columna matricula será nuestra clave principal de la tabla.

Analizando los datos vemos que:

- Dato matricula es alfanumérico, contiene números y letras.
- Dato nº de puertas es numérico, (y será ≥ 3 y ≤ 5).
- Dato Color es Alfabético.



¿Cómo haríamos un diagrama de la secuencia de venta de un coche?



Este es un diagrama básico para entender la lógica de programar, pero debe cumplir unas normas que iremos viendo a lo largo del tema.

1.2. Recordando conceptos



Fuete: Pxfuel

Una base de datos, es un conjunto organizado de datos y/o información.



"Es un conjunto exhaustivo no redundante de datos estructurados, organizados (independientemente de su utilización y su implementación), accesibles a tiempo real y que pueden ser leídos al mismo tiempo por diferentes usuarios (usuarios concurrentes), y que, realizando procesos sobre ellos, obtenemos diferentes informaciones en momentos no predecibles en el tiempo."

Es el elemento más importante del sistema de información y, debe de estar sujeto a medidas de seguridad (definidas en los procedimientos) que garanticen su integridad y eviten el acceso por personal no autorizado.

Deben cumplir unas determinadas normas, para que el uso de datos sea eficiente y el resultado fiable.

Forma un conjunto de archivos electrónicos. Las bases de datos tradicionales se organizan por campos, registros y archivos.

El diseño, desarrollo y gestión de bases de datos ha evolucionado mucho y, en la actualidad, es una parte esencial de cualquier entorno informático, por lo que es imprescindible tener conocimiento sobre ellas.



Recuerda

Ya has aprendido en unidades anteriores las estructuras de datos: ficheros, árboles, array...

1.2.1. Procedimiento

Un procedimiento, subrutina o subprograma es un fragmento del programa que realiza una tarea concreta y recibe un nombre por el que puede ser llamado o activado desde otra parte del programa (Prieto, 2006).

Un procedimiento puede tener argumentos, que son una serie de variables de comunicación, que permiten el paso de información entre el programa y el procedimiento.

1.2.2. Sistema Gestor de Base de Datos SGBD

Es un software que define y controla una estructura de datos, su almacenamiento, método de acceso y manipulación, tanto para el gestor de la base de datos como para los usuarios, manteniendo la integridad, confidencialidad y seguridad de dichos datos.



(Un software, sabemos que es un conjunto coordinado de programas, procedimientos, lenguajes, etcétera).

Su objetivo principal es simplificar y facilitar el acceso a datos, de forma rápida y fiable.

1.2.2.1. Tipos de bases de datos por clasificaciones

La elección de una base de datos depende de factores como la estructura de los datos, el volumen de información o los requisitos de escalabilidad. Existen múltiples clasificaciones, siendo la más fundamental la basada en el modelo de datos.

1.2.2.1.1. Clasificación por modelo de datos

El modelo de datos determina cómo se organiza y accede la información. Cada modelo ofrece ventajas específicas para distintos tipos de aplicaciones.

Modelo relacional

Organiza los datos en tablas compuestas por filas y columnas. Utiliza claves primarias y foráneas para establecer relaciones entre tablas y garantizar la integridad referencial. Es ideal para aplicaciones que requieren consistencia y transacciones ACID, como sistemas financieros o gestores de contenidos.

Modelo NoSQL

Las bases de datos no relacionales (NoSQL) surgieron como alternativa al modelo relacional para manejar grandes volúmenes de datos no estructurados. Se caracterizan por esquemas flexibles que no requieren estructuras fijas como tablas, y priorizan la escalabilidad horizontal.

A diferencia de los SGBDR, no siempre utilizan SQL como lenguaje principal y suelen optimizarse para patrones de acceso específicos, evitando operaciones complejas como JOINS para mantener el rendimiento distribuido. Si bien inicialmente relajaban las garantías ACID para lograr mayor escalabilidad, muchas bases NoSQL modernas (como MongoDB o Google Spanner) ofrecen actualmente transacciones ACID completas.

- **Modelo orientado a documentos:**

MongoDB es el principal exponente. Almacena la información en documentos flexibles que permiten estructuras complejas y anidadas en formatos como JSON o BSON. Cada documento puede tener una estructura independiente, lo que facilita la evolución del esquema: se pueden tener datos jerárquicos o relacionados dentro de un mismo documento.

Es ideal para contenidos jerárquicos, como catálogos de productos o perfiles de usuario, aplicaciones donde la flexibilidad del esquema es importante, como en aplicaciones web, análisis de datos y gestión de contenido.



- **Modelo clave-valor:**

Redis, Memcached son los ejemplos más conocidos de las bases de datos en memoria de clave valor. Almacenan datos en pares clave-valor, donde cada clave es única y se asocia a un valor. Evitar las estructuras complejas y que los datos se almacenan generalmente en memoria RAM permite un acceso extremadamente rápido.

Se usa principalmente para cachés, sesiones de usuario o configuraciones dinámicas.

- **Modelo de grafos:**

Estas bases de datos utilizan estructuras de grafos para almacenar y representar datos y sus relaciones, facilitando la navegación y consulta de relaciones complejas.

Representan los datos como nodos (entidades) y aristas (relaciones entre entidades). Es perfecto para redes sociales, sistemas de recomendación o detección de fraudes.

- **Modelo orientado a columnas:**

Almacena los datos por columnas en lugar de por filas, agrupando valores del mismo tipo. Esto acelera las consultas analíticas que procesan grandes volúmenes de datos. Es común en entornos de business intelligence o data warehousing.

Modelos históricos

Fueron populares antes de la llegada del modelo relacional. Aportaron ideas fundamentales pero carecían de la flexibilidad de los modelos modernos.

- **Modelo jerárquico:**

Organiza los datos en una estructura de árbol donde cada registro tiene un único padre. Es eficiente para datos con relaciones one-to-many predefinidas. Sin embargo, su rigidez lo hace difícil de adaptar a cambios en los requisitos.

- **Modelo en red:**

Extiende el modelo jerárquico permitiendo que un registro tenga múltiples padres. Facilita la representación de relaciones many-to-many sin duplicar información. Aun así, su complejidad lo relegó frente a la simplicidad del modelo relacional.

- **Modelo orientado a objetos:**

Almacena datos como objetos, similares a los usados en programación orientada a objetos. Preserva la herencia, el encapsulamiento y las relaciones entre objetos. Suele usarse en aplicaciones científicas o de diseño donde los datos son inherentemente complejos.



1.2.2.1.2. Clasificación por capacidades del sistema

Base de datos unimodelo

Una base de datos unimodelo es aquella diseñada para soportar un único modelo de datos de forma nativa. El término es útil para contrastarla con las bases de datos multimodelo, aunque en la práctica no se suele usar; simplemente se las denomina por su modelo (ej: "base de datos relacional"). La gran mayoría de las bases de datos tradicionales son unimodelo. Por ejemplo:

- MySQL es unimodelo (relacional), ya que se optimiza para trabajar con tablas y SQL.
- MongoDB es unimodelo (documentos), diseñado específicamente para almacenar y consultar documentos BSON.

Un sistema unimodelo suele ofrecer un rendimiento y una optimización superiores para su modelo específico, al estar especializado.

Base de datos multimodelo

Una base de datos multimodelo es un sistema integrado capaz de gestionar datos utilizando múltiples modelos de forma simultánea sobre un mismo motor de almacenamiento. A diferencia de las bases de datos unimodelo, permite trabajar con datos relacionales (tablas), documentos (JSON), grafos y otros modelos dentro de una misma plataforma.

El principal beneficio es la flexibilidad y la reducción de la complejidad operativa. Elimina la necesidad de mantener varias bases de datos especializadas para una misma aplicación (evitando el "poligloteismo de persistencia"), simplificando el desarrollo y la administración. Ejemplos destacados son PostgreSQL (que extiende el modelo relacional con JSONB y extensiones para grafos) y ArangoDB (nativo multimodelo para documentos, grafos y clave-valor).

1.2.2.1.3. Clasificación por propósito

Esta categorización prioriza el propósito de la base de datos sobre su estructura interna. ayuda a seleccionar sistemas optimizados para cargas de trabajo específicas.

Bases de datos transaccionales (OLTP)

Manejan operaciones frecuentes y cortas, como inserciones, actualizaciones o consultas puntuales. Garantizan las propiedades ACID para asegurar la integridad de cada transacción. Son propias de sistemas de comercio electrónico, reservas o banca.

Bases de datos analíticas (OLAP)

Se especializan en consultas complejas que analizan grandes volúmenes de datos históricos. Su estructura está optimizada para operaciones de agregación y generación de informes. Forman el núcleo de herramientas de business intelligence y data mining.



1.2.2.1.4. Bases de datos especializadas

Están diseñadas para tipos de datos o dominios de aplicación muy concretos. Su diseño optimiza rendimiento y funcionalidades para casos de uso específicos.

Bases de datos geográficas (GIS)

Manejan datos espaciales como coordenadas, mapas o formas geométricas. Incluyen operaciones especializadas para calcular distancias, áreas o intersecciones. Son esenciales en sistemas de navegación, urbanismo o gestión de recursos naturales.

Bases de datos de series de tiempo

Almacenan puntos de datos indexados por tiempo, como lecturas de sensores o cotizaciones bursátiles. Comprimen datos eficientemente y optimizan consultas por rangos temporales. Se usan en monitorización industrial, finanzas o IoT.

Bases de datos en memoria

Almacenan los datos primariamente en memoria RAM para minimizar latencias de acceso. Logran velocidades extremadamente altas en operaciones de lectura y escritura. Son ideales para cachés, sesiones de usuario o aplicaciones en tiempo real.

1.3. Objetivos y cualidades de un buen diseño

Un buen diseño debe definir cosas primordiales como la estructura de datos, tipo de dato, si es o no un dato obligatorio etc. Por ello los objetivos del diseño son:

- Obtener un resultado que **satisfaga los requisitos del software** (y por lo tanto nuestras necesidades o las de nuestros clientes).
- Ser **fácil de mantener**.
- **Evitar problemas**, como pueden ser:
 - **Redundancia:**
Repetición de información (información duplicada).
 - **Inconsistencia:**
Existe información contradictoria o incongruente.
 - **Dificultad de acceso a los datos:**
Si el formato de la información no es uniforme, habría que realizar una conversión y combinación de los datos de distintas tablas o archivos.



- **Acceso concurrente no controlado:**

No se tienen los mecanismos adecuados para la sincronización de procesos que acceden simultáneamente a la base de datos.

- **Problemas de seguridad:**

Posibilidad de accesos por parte de personal no autorizado.

- **Dificultad en el procesamiento de datos:**

Puede ocurrir que, debido al formato, no pueda ser utilizado por otras herramientas como, por ejemplo, herramientas de generación de informes.

- **Integridad:**

Es muy importante, debe evitar:

- » Datos duplicados.
- » Datos faltantes.
- » Datos alterados.
- » Datos incorrectos.



Atención

Integridad:

Si un dato está en varias tablas (o ficheros) y se modifica en una de ellas, (modificamos el DNI en la tabla de empleados), este debe modificarse en todas las tablas o ficheros donde aparezca).

Según Thomas H. Grayson, un buen diseño de base de datos debe poseer las siguientes **cualidades**:

- Reflejar la estructura del problema en el mundo real.
- Ser capaz de representar todos los datos esperados, incluso con el paso del tiempo.
- Evitar el almacenamiento de información redundante.
- Proporcionar un acceso eficaz a los datos.
- Mantener la integridad de los datos a lo largo del tiempo.
- Ser claro, coherente y de fácil comprensión.



1.4. Fases del diseño de base de datos. Esquematización

El objetivo de dividir el diseño en fases, es conseguir un esquema físico que, posteriormente, se plasmará en un SGBD dando lugar a nuestra base de datos.

Para diseñar una base de datos, partimos de **la especificación de requisitos del usuario** (aquí se indican las necesidades que hay que cubrir).

La especificación de requisitos es un paso fundamental, nuestro cliente, nos solicita un programa que debe cubrir las necesidades concretas de los usuarios que lo utilizarán, y el programa que creemos debe satisfacer totalmente.

Para ello es imprescindible:

- Entender perfectamente lo que el usuario nos pide.
- A partir de lo que nos pide, concluir otras necesidades que también necesita, aunque no sea consciente de ellas. Entender lo que el usuario no puede pedir.
- Añadir cosas que el usuario no ha pedido o aún no necesita, pero que sabemos que necesitará más adelante.

Es muy importante que el cliente y los usuarios participen durante todo el proceso para asegurar que al final se consiga lo que necesitan.



Atención

Los requisitos definen qué debe hacer un sistema (el problema), mientras que el diseño define cómo hacerlo (la solución).

A partir de la especificación de Requisitos se comienza el trabajo en las diferentes fases del diseño para lograr una buena Base de Datos, que son:

- **Fase 1:** Diseño conceptual.
- **Fase 2:** Diseño Lógico.
- **Fase 3:** Diseño Físico.



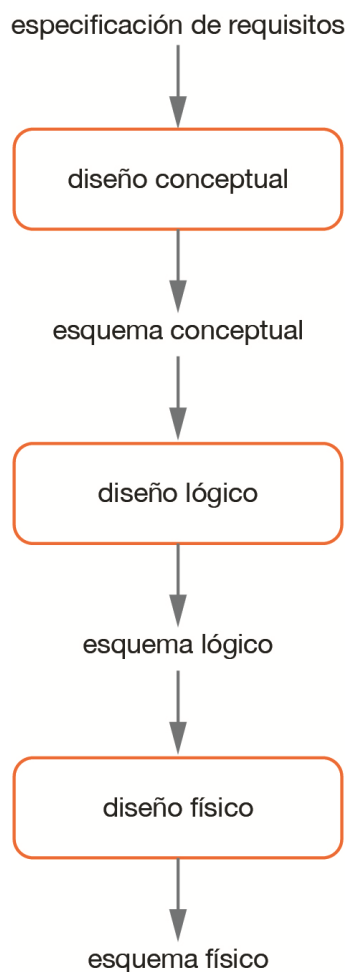
Dependencia de cada fase con el SGBD

En la siguiente tabla puedes observar la dependencia de cada una de las fases de diseño con el tipo de sistema gestor de base de datos y con un SGBD específico.

Fase	Tipo de SGBD	SGBD específico
Diseño conceptual	No	No
Diseño lógico	Sí	No
Diseño físico	Sí	Sí

Esquematización del diseño de Bases de Datos en fases

Los pasos son los siguientes:





2. Fase 1. Diseño conceptual

Ya has visto, que consiste en, a partir de la especificación de requisitos, crear un Esquema Conceptual, que se utilizara para la siguiente fase, el Diseño Lógico.



Etapa del diseño conceptual. Entradas y salidas

El diseño conceptual comienza a la finalización de la especificación de requisitos, tiene por objeto el descubrimiento y comprensión de la semántica del sistema y su materialización en un esquema conceptual, construido en base a un modelo conceptual.

Para ello se utiliza un modelo conceptual, que puede ser el Modelo Entidad/Relación -MER(el más usual), el orientado a objetos...

El objetivo del diseño conceptual es crear el esquema conceptual, que se representa típicamente (si hemos usado MER) en un Diagrama Entidad-Relación (DER). Este esquema define la estructura de la información (entidades, atributos y relaciones) de forma independiente de cualquier SGBD, todavía lejos de la implementación física.



Imprescindible

El diseño conceptual se centra en identificar las entidades, sus atributos principales y las relaciones entre ellas.

Se trabaja con un modelo conceptual, que no entra en detalles como dominios de atributos, tipos de datos ni restricciones de integridad complejas.

Representa los elementos que intervienen en un problema y sus relaciones.

El más utilizado, como dijimos, es el Modelo Entidad-Relación (Peter Chen en 1976).



2.1. Esquema conceptual

Descripción abstracta y de alto nivel de la estructura de una base de datos, independiente de cualquier tecnología o Sistema Gestor de Bases de Datos (SGBD). Define qué datos se almacenan (entidades, atributos) y las relaciones entre ellos, sin especificar cómo se implementarán físicamente. Se representa típicamente mediante Diagramas Entidad-Relación (DER) y sirve como punto de partida fundamental para el diseño lógico.

El esquema conceptual es la culminación del diseño conceptual, representando de forma gráfica y unificada la estructura de los datos. Es el producto final de esta fase.

La metodología de diseño conceptual suele comenzar identificando las distintas vistas de los usuarios (esquemas conceptuales locales). Posteriormente, estas vistas se integran y consolidan en un esquema conceptual global que representa de manera coherente todos los requisitos de la organización.

2.1.1. El modelo conceptual

El modelo conceptual es el lenguaje o marco formal que utilizamos para representar el esquema conceptual de una base de datos. Proporciona las herramientas necesarias para describir una realidad de manera abstracta, obteniendo como resultado un esquema conceptual gráfico.

Nos ofrece unas herramientas, para describir una realidad mediante una representación gráfica, obteniendo un esquema conceptual.

Modelo conceptual: **herramientas** → esquema conceptual (producto gráfico).

Estas herramientas del modelo conceptual, de una forma gráfica, nos tienen que permitir y servir una serie de conceptos:

- **Permitirnos las propiedades:**
 - Expresividad (poder describir la realidad necesaria).
 - Simplicidad (poder describir la realidad necesaria).
 - Minimalidad (evitar elementos redundantes).
 - Formalidad (tener reglas bien definidas).
 - No tiene en cuenta el lenguaje de programación que utilizaremos.
- **Servirnos para:**
 - Describir datos y las relaciones entre ellos.
 - Capturar la semántica de los datos.
 - Definir a nivel semántico las restricciones de consistencia.



En el diseño conceptual, las restricciones de consistencia se expresan mediante reglas semánticas y estructurales en el modelo Entidad-Relación (DER).

Restricciones de:

- **Integridad de clave:** garantizan que cada instancia de una entidad sea única e identificable (*el DNI de un Cliente debe ser único y no nulo*).
- **Integridad de entidad:** establecen la participación obligatoria de una entidad en una relación (*para poder tener la condición de alumno se ha de estar matriculado como mínimo en una asignatura*).
- **Cardinalidad:** definen los límites numéricos de una relación entre entidades (1:1, 1:N, N:M), lo estudiaremos en detalle un poco más adelante en esta misma unidad.
- **Dominio o valor:** definen el conjunto de valores válidos que puede tomar un atributo (*la Edad de una entidad persona ha de ser un entero mayor que 0...*).
- **Semánticas o reglas de negocio:** las más específicas y dependen completamente del contexto del problema. Capturan lógica compleja que no se puede expresar solo con cardinalidades o dominios (*la fecha de fin de contrato no puede ser anterior a la del inicio...*).

Estas restricciones garantizan que el esquema conceptual refleje con precisión las reglas del mundo real, sirviendo como base para su posterior implementación técnica en el diseño lógico.



Resumiendo

El modelo conceptual es un conjunto de herramientas gráficas (conceptuales) para describir datos, sus relaciones, su significado y sus restricciones de consistencia, en forma de esquemas conceptuales (forma visual).

2.1.2. Modelo entidad-relación (E/R) de Peter Chen (1976)

El Modelo Entidad-Relación (E/R) de Peter Chen es una herramienta de modelado de datos de alto nivel que permite representar gráficamente la estructura lógica y las reglas de negocio de un sistema. Utiliza una notación específica (entidades, atributos, relaciones y cardinalidades) para crear un esquema conceptual que es independiente de la tecnología de base de datos que se vaya a utilizar posteriormente.

El objetivo es, de una forma gráfica, representar la descripción de datos, relaciones entre datos, semántica de los datos y restricciones de consistencia, creando una visión real y natural (mediante entidades e interrelaciones).



El modelo original utilizaba 3 conceptos:

- **Entidad.**
- **Relación entre entidades.**
- **Atributo.**
- **Cardinalidad.** Como concepto importante, para determinar los tipos de relación entre entidades.

Más adelante, también se han ido añadiendo nuevos conceptos para mejorar su capacidad expresiva, convirtiéndolo en modelo Entidad/Relación extendido mediante:

- Dominios de atributos.
- Identificadores.
- Atributos compuestos.
- Jerarquías de generalización/especialización.

2.1.2.1. Entidad (entity). Tipos

Una entidad representa 'algo' relevante para el dominio del negocio o sistema con existencia independiente y única, cuya información necesita ser almacenada.

Es un tipo de objeto sobre el que se recoge información. Puede ser una persona, una cosa, un concepto o un suceso.

Cada tipo de entidad debe tener un nombre único en el esquema.



CONVO 2022

En el examen de la convocatoria del 22 se decía que Métrica v3 entendía Entidad en el Modelo Entidad-Relación Extendido, como "aquel objeto, real o abstracto, acerca del cual se desea almacenar información en la base de datos".

Las entidades deben tener atributos que describan sus características. Por ejemplo, un Gato tiene atributos como Nombre, Color, Edad, un Coche puede tener atributos como Marca, Modelo, etc.



Tipos de Entidades

En el modelo E/R, las entidades pueden clasificarse según su dependencia existencial:

- **Entidad fuerte (regular).**

Objeto con existencia independiente en el sistema (puede ser físico o conceptual: persona, departamento, proyecto...). Sus ocurrencias no dependen, para existir, de la presencia de ocurrencias en ninguna otra entidad.

Se representa gráficamente con un rectángulo con el nombre de la entidad en el interior (el color de relleno no es necesario).



- **Entidades débil (dependiente).**

La entidad débil se caracteriza por necesitar de la existencia de la entidad fuerte para existir. Su clave primaria combina un atributo propio con la clave primaria de la entidad fuerte de la que depende.

El ejemplo perfecto es el de un sistema de ventas donde Factura es la entidad fuerte y Línea_Factura es la entidad débil. La entidad Factura tiene atributos como N°_Factura, Fecha, Cliente, Total, etc. La entidad Línea_Factura tiene como atributos N°_Factura (que referencia a la entidad fuerte), N°_Línea, Descripción_Producto, Precio y Cantidad. El identificador principal de Línea_Factura sería (N°_Factura, N°_Línea), ya que una línea de factura solo se identifica de manera única en el contexto de la factura a la que pertenece.

Según la notación clásica (Chen), se representa la entidad débil con un rectángulo contenido dentro de otro o rectángulo doble.



La debilidad es una característica opcional que aparece solo cuando modelas relaciones de dependencia existencial.



2.1.2.2. Atributo (attribute)

Atributo es una característica o propiedad de un tipo de entidad.

Los atributos son la unidad básica de información que sirve para describir las características de la entidad; "para identificar la entidad".

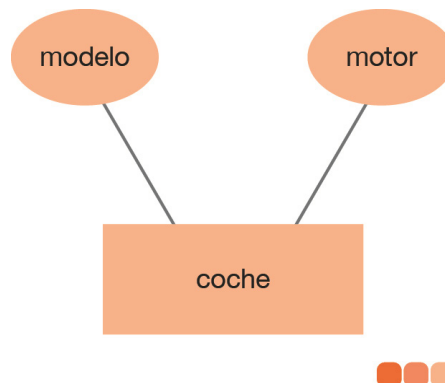
(Puede ser un número elevado, el diseñador decidirá cuales indicar, los más relevantes para el sistema).

Cada entidad debe contener un número mínimo de distintos atributos, que la identifiquen inequívocamente.

Los atributos pueden ser de distintos tipos (numéricos, texto, etcétera).

Tanto las relaciones como las entidades pueden tener atributos.

Se representan como elipses conectadas a la entidad por una línea.



Cada entidad se diferencia de las demás por el valor de su identificador.

Dos o más entidades no pueden tener el mismo valor en todos los atributos que forman el identificador.

Restricciones sobre atributos

Si defines restricciones sobre los atributos, estos pueden ser restringidos por:

- **Multiplicidad de valores:**
 - **Univaluado:** cada instancia de la entidad puede tener solo un valor para este atributo (una persona solo puede tener una fecha de nacimiento).
 - **Multivaluado:** cada instancia de la entidad puede tomar varios valores para este atributo (una persona puede tener más de un número de teléfono).



- **Obligatoriedad:**
 - **Obligatorio:** el atributo debe contener un valor en todas las instancias de la entidad (DNI de una persona).
 - **Opcional:** el atributo puede carecer de valor (ser nulo) en algunas instancias (segundo apellido de una persona).

Atributos identificador



Ejemplo

En el ejemplo de los coches, los atributos de la entidad "coche" serían:

- **Modelo.** Ejemplo: Ford Mondeo (tipo texto).
- **NPuertas.** Ejemplo: 5 (tipo numérico entero).
- **Color.** Ejemplo: gris (tipo texto).
- **Motor.** Ejemplo: gasolina (tipo texto).

Este ejemplo muestra atributos descriptivos, pero falta especificar el atributo identificador. Para la entidad **coche**, el identificador principal sería normalmente la **matrícula**.

Atributo Identificador de una Entidad (Identificadores)

El identificador de una entidad es un atributo o conjunto de atributos que identifica unívocamente a cada ocurrencia de esa entidad.

El identificador de una entidad debe cumplir tres condiciones:

- No pueden existir dos ocurrencias de la entidad con el mismo valor del identificador.
- No se puede omitir ningún atributo identificador (no pueden contener un valor nulo).
- Toda entidad tiene al menos un identificador y puede tener varios identificadores secundarios, alternativos o candidatos.



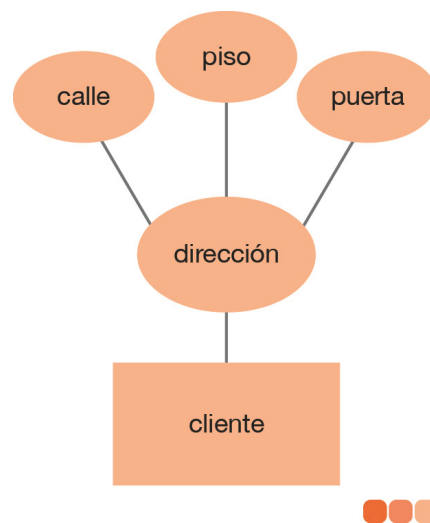
+Info

En el modelo E/R conceptual, es válido identificar posibles atributos que podrían servir como identificadores (ej: DNI, email para CLIENTE), pero la elección del identificador principal y el concepto de "claves candidatas" pertenecen más al modelo lógico-relacional.

Atributos Compuestos

Son aquellos que se pueden dividir en subpartes más pequeñas, que representan atributos más básicos con significado propio.

Ejemplo:



Ejemplo de atributo compuesto

Para cada atributo, definiremos el tipo de valor que puede tomar, restringiendo así el tipo de dato que almacenaremos. (sólo cadenas de caracteres, sólo números, solo números mayores que cero... en el caso de atributo de nº de teléfono de una entidad persona, exigiremos que sean 9 números).

Atributos Atómicos

Los atributos que no son compuestos, no pueden dividirse, se denominan atómicos o de valor simple.

Son los que tienen un solo valor para una entidad particular. No pueden dividirse (no son compuestos).



Dominio de Atributos (values set)

Dominio de un atributo es el conjunto de todos los posibles valores que puede tomar el atributo (el tipo de datos).

Por ejemplo, un tipo de dato "entero" podrá tomar todos los valores enteros (1, 2, 3, etcétera) pero no otro tipo de valores como decimales, letras, fechas... Un tipo de dato lógico, solo podrá tomar el valor verdadero o falso.

También definiremos si es obligatorio o no, que exista un valor para ese atributo.

Cuando un atributo no tiene un valor determinado, recibe el valor nulo.



Ejemplo de Dominios de atributos

El **atributo "kilómetros"** puede tomar un valor entero positivo. Si establecemos que no se venderán coches con más de 500.000 km, el dominio sería [0-500000] (el conjunto de los números enteros comprendidos entre 0 y 500.000).

El **atributo "motor"** puede tomar varios valores específicos. Su dominio de atributos sería la lista: {"gasolina", "diésel", "eléctrico", "híbrido"}.

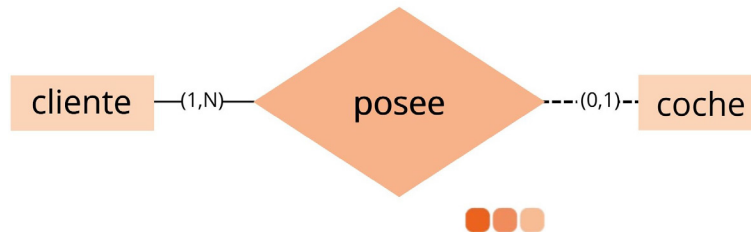
2.1.2.3. Relación entre entidades

En un modelo Entidad-Relación (E/R), una **relación describe la correspondencia o asociación entre dos o más entidades**. La relación asimismo puede tener atributos propios.



Ejemplo

Imaginemos una base de datos de una empresa de venta de coches. Dos de las entidades podrían ser Cliente y Coche. En este caso, podríamos tener una relación llamada "Posee" para indicar qué coches son propiedad de qué clientes.



- **Tipo de Relación:**

La relación entre Cliente y Coche sería de tipo uno a muchos (1:N). Esto significa que un cliente puede poseer varios coches, pero cada coche solo puede pertenecer a un único cliente.

- **Participación:**

- Del lado del Cliente podría ser **total** (todo cliente debe estar asociado obligatoriamente a un coche).
- Del lado del Coche podría ser **parcial** (un coche puede existir en el sistema sin tener aún un cliente).

Las relaciones, en el modelo E/R, se representan gráficamente mediante rombos con el nombre en el interior.

Métrica 3 y el modelo E/R

Es una metodología de planificación, desarrollo y mantenimiento de sistemas de información, propiedad del Ministerio de Administraciones Públicas de España. Es de uso libre, requiriendo únicamente la citación de su fuente oficial. Esta metodología proporciona a las organizaciones un marco sistemático para gestionar el ciclo de vida completo del software.

Dentro de su estructura, Métrica 3 emplea el Modelo Entidad-Relación (E/R) como herramienta fundamental durante la Fase de Diseño del Sistema, específicamente en la etapa de Diseño de Datos. Su objetivo es crear el Modelo de Datos Conceptual y el Modelo de Datos Lógico.

Conceptos Clave del Modelo E/R en Métrica 3:

- **Relación:** es la asociación que existe entre dos o más conjuntos de entidades. Se representa gráficamente con un rombo.
- **Participación:** define si la asociación es obligatoria (participación total) u opcional (participación parcial) para una entidad.



2.1.2.3.1. Clasificación de las Relaciones en el Modelo E/R

Las relaciones en el modelo Entidad-Relación se pueden caracterizar según dos aspectos independientes:

Según el GRADO de relación

El grado indica el número de conjuntos de entidades que participan en una relación.

- **Binaria:** es el más frecuente, grado 2 (*Cliente* - *Compra* - *Producto*).
- **Ternaria:** grado 3, relaciona tres entidades (*Paciente* - *Es tratado por* - *Médico* - *Enfermedad*).
- **Reflexiva o recursiva:** grado 1, entidad se relaciona consigo misma (*Empleado* - *Supervisa* - *Empleado*).

Según la PARTICIPACIÓN de las entidades

Indica si la participación de una entidad en una relación es obligatoria u opcional.

- **Total:** Todas las entidades del conjunto deben participar en la relación. Esta relación también puede ser nombrada según su obligatoriedad, en este caso sería de **Obligatoriedad u Obligatoriedad 1**.

En nuestro ejemplo "posee" entre Cliente y Coche, si para ser cliente DEBE de tener un coche la participación del cliente es TOTAL.

- **Parcial:** Algunas entidades del conjunto pueden no participar en la relación, relación que también podríamos denominar como de **No obligatoriedad u Obligatoriedad 0**.

Por lo contrario, en este mismo ejemplo, sabemos que es parcial, pues no todo coche ha de haber sido comprado, dicho de otra manera, **no todo coche ha de tener un cliente**.

2.1.2.4. Relaciones modelo extendido

En el modelo Entidad/Relación extendido, se definen las relaciones por:

- **Nombre.**
- **Cardinalidad.**
- **Tipo de correspondencia.**



2.1.2.4.1. Nombre

El nombre de la relación la distingue unívocamente del resto de relaciones del modelo.

2.1.2.4.2. Cardinalidad

En el modelo Entidad–Relación es fundamental distinguir entre la determinación de la cardinalidad de una relación y la forma en que dicha cardinalidad se expresa mediante una notación estándar, como es el caso de 1:N. Aunque ambos conceptos están relacionados, no son equivalentes y conviene tratarlos de forma separada.

Determinación de la cardinalidad de una relación.

La cardinalidad de una relación se determina a partir de la participación de las entidades en dicha relación. Para cada entidad se establece una dupla de participación (mínimo, máximo), que indica el número mínimo y máximo de veces que una ocurrencia de la entidad puede intervenir en la relación.

Una vez determinadas las duplas de participación de todas las entidades implicadas, la cardinalidad de la relación se obtiene considerando los valores máximos de dichas participaciones. Es decir, la cardinalidad expresa cuántas ocurrencias de una entidad pueden asociarse, como máximo, con una ocurrencia de la otra entidad a través de la relación.

Por tanto, la cardinalidad no depende del sentido de lectura de la relación ni de la forma en que se describa verbalmente, sino únicamente de los límites máximos de participación de las entidades. La información completa sobre el comportamiento de la relación se recoge siempre en las duplas de participación, siendo la cardinalidad una síntesis de dichas participaciones máximas.

2.1.2.4.3. Tipo de Correspondencia

El tipo de correspondencia entre entidades indica cómo se relacionan las ocurrencias de unas entidades con las de otras dentro de una relación. Este concepto se utiliza para clasificar las relaciones en función del número máximo de ocurrencias que pueden asociarse entre sí las entidades implicadas.

El tipo de correspondencia se determina a partir de la cardinalidad máxima de la relación, y permite identificar si una relación es de uno a uno, de uno a varios o de varios a varios. Por tanto, no describe la obligatoriedad de participación ni los mínimos, sino únicamente el tipo de asociación máxima existente entre las entidades.

En un conjunto de relaciones en el que participan dos o más entidades, el tipo de correspondencia indica con cuántas ocurrencias de una entidad puede relacionarse, como máximo, una ocurrencia de la otra entidad.



Principales tipos de correspondencia:

- **Relaciones Uno a Uno (1:1).**

En una relación uno a uno, una ocurrencia de una entidad puede relacionarse, como máximo, con una única ocurrencia de la otra entidad, y viceversa. Este tipo de correspondencia indica una asociación directa entre ambas entidades, aunque no implica necesariamente que la participación sea obligatoria en ambos sentidos. La obligatoriedad se determina, en su caso, mediante las duplas de participación.

- **Relaciones Uno a Varios 1:N.**

En una relación uno a varios, una ocurrencia de una entidad puede relacionarse con varias ocurrencias de la otra entidad, mientras que cada ocurrencia de esta última solo puede relacionarse con una ocurrencia de la primera.

Una vez determinada la cardinalidad de la relación a partir de las participaciones máximas, esta se representa mediante la notación estándar 1:N. Esta notación responde a una convención de escritura, cuyo objetivo es unificar la forma de representar las cardinalidades. El valor 1 corresponde a la entidad cuya participación máxima es uno, mientras que el valor N corresponde a la entidad cuya participación máxima es muchos.

Aunque conceptualmente la relación pueda describirse desde cualquiera de las entidades implicadas, la cardinalidad es única y no varía. La notación 1:N no implica una elección ni depende del punto de partida, sino que representa de forma normalizada una cardinalidad previamente determinada.

Para describir con mayor precisión el comportamiento de las entidades en la relación, especialmente en lo relativo a la obligatoriedad de participación, es necesario recurrir a las duplas de participación (mínimo, máximo), ya que la notación 1:N únicamente refleja los valores máximos y no aporta información sobre los mínimos.

- **Relaciones Varios a Varios (N:M).**

En una relación varios a varios, una ocurrencia de una entidad puede relacionarse con varias ocurrencias de la otra entidad, y viceversa. Este tipo de correspondencia indica que no existe una limitación máxima de uno en ninguno de los dos sentidos de la relación.

Al igual que en los casos anteriores, el tipo de correspondencia N:M se determina a partir de las participaciones máximas de las entidades implicadas, mientras que los mínimos de participación se establecen mediante las correspondientes duplas.

2.1.2.5. Jerarquías de generalización/especialización

Una entidad A es una generalización de un grupo de entidades ($A_1, A_2, \dots, A_n$) si cada ocurrencia de cada una de esas entidades es también una ocurrencia de A.

Las entidades $A_1, A_2, \dots, A_n$ serán, a su vez, especializaciones de la entidad A.

Todas las propiedades de la entidad genérica A son heredadas por las sub-entidades.



Ejemplo

La entidad "empleados" sería una generalización de las entidades "vendedores" y "mecánicos".

La entidad "mecánicos" sería una especialización de la entidad "empleados".

Cada jerarquía es, por un lado, total o parcial, y por el otro, exclusiva o superpuesta.

- **Jerarquía total.**

Cada ocurrencia de la entidad genérica corresponde al menos con una ocurrencia de alguna subentidad.

- **Jerarquía parcial.**

Alguna ocurrencia de la entidad genérica no corresponde con ninguna ocurrencia de las subentidades.

- **Jerarquía exclusiva.**

Cada ocurrencia de la entidad genérica corresponde, como mucho, con una ocurrencia de una de las subentidades.

- **Jerarquía superpuesta.**

Alguna ocurrencia de la entidad genérica corresponde a ocurrencias de dos o más subentidades diferentes.

Un subconjunto es un caso particular de generalización con una sola entidad como subentidad. Un subconjunto siempre es una jerarquía parcial y exclusiva.

2.2. Construcción del modelo conceptual de datos. Metodología

La metodología nos indica unos pasos a seguir.

Sirve de guía, pero no es rígida, es adaptable a nuestras necesidades.

Hay muchas variantes dependiendo del autor. La iremos adaptando, pudiendo repetir alguno de los pasos (o parte de ellos) de forma iterativa cuando sea necesario.



Los pasos propuestos son:

1. Recopilar información.
2. Identificar entidades.
3. Identificar las relaciones entre entidades.
4. Identificar atributos.
5. Determinar los dominios de los atributos.
6. Determinar los atributos clave candidata, principal y alternativa.
7. Identificar generalizaciones/especializaciones.
8. Identificar las entidades débiles.
9. Comprobar si hay redundancia.
10. Validar el modelo conceptual para ver si cumple los requisitos.
11. Repasar el modelo conceptual con los usuarios.
12. Generar documentación.

1. Recopilar información

Recopilar información: Especificación de Requisitos

Es el primer paso, recoger la información relevante del universo que se quiere representar.

Debemos conocer toda la información necesaria para lograr los objetivos de la programación a realizar. El usuario debe indicarnos todos los datos que utiliza, así como cuáles son sus necesidades. Añadiendo también las que nosotros preveamos que necesitará.

2. Identificar entidades

Identificamos los objetos que tengan existencia propia

Definimos los objetos principales en los que los usuarios están interesados, les asignamos un nombre.



3. Identificar las relaciones entre entidades

Asignar nombre a las relaciones

Les asignamos un **nombre a las relaciones** de las entidades definidas en el punto anterior: **verbos** o expresiones verbales.

4. Identificar atributos

Identificar características posibles

Buscamos las características de cada entidad, y les asignamos un nombre (nombre del atributo).

5. Determinar los dominios de los atributos

Decidir valores y su rango

Definimos todos los posibles valores (y rango de valores) que puede tomar el atributo (el tipo de datos).

6. Determinar los atributos clave candidata, principal y alternativa

Escogemos la clave principal y si lo consideramos necesario también las alternativas

Determinamos que atributos o conjuntos de atributos son claves candidatas (pueden identificar unívocamente una ocurrencia de la entidad).

7. Identificar generalizaciones/especializaciones

Ya hemos visto los conceptos de generalización y especialización, pero vamos a recordarlo

La especialización es el proceso para clasificar una clase de objetos en subclases más especializadas.



La generalización es el proceso inverso. Se generalizan varias clases para obtener una abstracta de más alto nivel que incluya los objetos de todas estas clases.

Los atributos identificadores y los descriptores comunes a todas las entidades estarán en la entidad general y el resto en las entidades especializadas.

8. Identificar las entidades débiles

Especificar las relaciones entre las entidades

Determinamos si las entidades son entidades fuertes (independientes) o débiles (dependientes).

Una entidad débil sufre dependencia de identificación.

Para identificarse necesita la ocurrencia de otras entidades con las que está relacionada. Debemos especificar la relación o relaciones que identifican a cada entidad débil.

9. Comprobar si hay redundancia

No debe haber redundancia

Tras construir el modelo entidad/relación, debe ser analizado para comprobar si se presentan redundancias.

Los atributos redundantes, que se derivan de otros elementos mediante algún cálculo, deben ser eliminados o marcarse como redundantes.

Hay que estudiar detenidamente las cardinalidades mínimas de las entidades, y la semántica de las relaciones. Las relaciones redundantes deben eliminarse del modelo, asegurándose que eliminándolas sigue siendo posible el paso, tanto en un sentido como en el inverso, entre las dos entidades que unían.

Debemos eliminar las relaciones o entidades redundantes. Las entidades redundantes se funden en una sola. La clave principal de una de ellas pasa a ser clave principal de la nueva entidad y la clave principal de la otra pasa a ser clave alternativa de la nueva entidad.



Ejemplo

En un hotel, las entidades "huésped" y "cliente" serían redundantes.



10. Validar el modelo conceptual para ver si cumple los requisitos

Repasar el trabajo realizado

Antes de repasar el modelo con el usuario, es importante repasarlo, para garantizar que se cumplen los requisitos.

11. Repasar el modelo conceptual con los usuarios

Mostrar el trabajo realizado a los usuarios, nuestro objetivo es que lo entiendan

Los usuarios deben estar implicados en este proceso y deben entender y validar que lo representado en el modelo conceptual es lo que ellos necesitan.

Si no entienden el modelo, generalmente, habrá que revisarlo hasta que se cumpla este requisito.

12. Generar documentación

Imprescindible tener el trabajo bien documentado

La documentación servirá de soporte para posteriores etapas de diseño. Toda la información recopilada queda definida en el diccionario de datos.

El diccionario de datos puede incluir:

- El modelo gráfico utilizado (por ejemplo, el modelo E/R).
- El esquema de base de datos.
- Catálogo de requisitos.
- Especificación del problema.
- Descripción de cada elemento del modelo (entidades, relaciones y atributos).
- Dominio de los atributos.
- Cualquier documentación adicional que el diseñador considere oportuna.
- Diagramas de flujo de datos. Reglas de construcción.



En el diseño conceptual el objetivo es capturar la semántica de los datos del sistema. Para ello, el Modelo Entidad-Relación (MER) es imprescindible, ya que permite definir entidades, atributos y relaciones de manera independiente de cualquier implementación técnica.

Como complemento a esta visión estructural de los datos, puede utilizarse el **Diagrama de Flujo de Datos (DFD)**, que, aunque no es obligatorio en el diseño conceptual de una base de datos, resulta muy útil para representar cómo circula la información y cómo se transforman los datos mediante los procesos del sistema, aportando una visión funcional que completa al MER.

2.3. Diagramas de flujo de datos

Un diagrama de flujo de datos (DFD) es una representación gráfica que muestra de forma clara y lógica los procesos del sistema de información.

Para ello se realiza una descomposición sucesiva de los diferentes procesos, desde el nivel más general hasta el nivel de detalle suficiente, mostrando la relación entre ellos.

El DFD establece las funciones que hay que desarrollar sin indicar cómo hacerlo.

Es independiente de las restricciones físicas del entorno.

Objetivo

El objetivo principal es simplificar el entendimiento y mantenimiento del sistema.

Proporcionar una representación del sistema a nivel lógico y conceptual que facilite su comprensión al equipo de desarrollo y a los usuarios.

Permite representar gráficamente:

- El flujo o movimiento de los datos a través del sistema.
- La lógica de los procesos.
- Las transformaciones de los datos al pasar por un proceso.
- Los límites del sistema.

El DFD, establece las funciones que hay que desarrollar, sin indicar cómo hacerlo.



Características

Un diagrama de Flujo de Datos, debe cumplir unas determinadas características, para que sea útil y de fácil comprensión.

La información debe ser:

- **Sintética.**

Debe ser resumido, de forma que un proceso debe verse en una sola hoja.

- **Simbolizada.**

Hay que utilizar unos símbolos concretos para cada elemento, de esta forma, visualmente, el símbolo utilizado ya nos proporciona rápidamente información.

- **Gráfica.**

De una forma visual, podemos ver rápidamente todos los procesos, mostrando unos rasgos principales, sin necesidad de leer notas. A su vez, con mayor atención podemos ver detalles.

Como analista, debes asegurar:

- Que ha desarrollado todas las partes del procedimiento.
- Que sirve para escribir un informe lógico con claridad.
- Que garantiza que, al mostrarlo al usuario, entienda los requisitos que solicito y cumple sus objetivos.

2.3.1. Elementos. Notación

Ya hemos indicado que debemos descomponer los procesos desde un nivel general, hasta el máximo nivel de detalle necesario para reflejar toda la semántica del sistema.

Las conexiones permitidas entre los elementos de un DFD son:

- ENTIDAD – PROCESO.
- PROCESO – PROCESO.
- PROCESO – ALMACÉN.



Notación

El diagrama de flujo de datos se compone de los siguientes elementos:

- **Almacén de datos:**

- Representa una colección de datos en reposo, almacenada, y que es controlada por el sistema de gestión de datos.
- Deben estar todos los datos que se necesitará en la ejecución de procesos.
- No puede crear, transformar ni destruir datos.
- No está comunicado con otro almacén o entidad externa.
- Aparecerá por primera vez en el nivel en que dos o más procesos accedan a él.

- **Entidad externa:**

- Representa un ente ajeno al sistema (personas, organizaciones u otros sistemas) que proporciona o recibe información de este.
- Las relaciones entre entidades externas no se estudian en el modelo.
- Normalmente solo aparece en el diagrama de contexto, aunque puede aparecer en niveles inferiores si ayuda a la comprensión del diagrama.
- Puede repetirse en un mismo nivel diagrama para no entrecruzar líneas.
- Nos proporciona información de la relación del sistema con el mundo exterior.

- **Proceso:**

- Muestra una funcionalidad que debe realizar el sistema para transformar o manipular los datos de entrada, generando datos de salida.
- El proceso no puede ser el origen último ni el final último de los datos.
- Una entidad externa y un almacén de datos solamente se pueden relacionar a través de un proceso (no directamente).
- Un proceso puede transformar un flujo de datos en varios.
- Los procesos representan transformaciones lógicas de datos y no describen control de ejecución, inicio ni finalización.

Un proceso, representa una función que tiene que realizar el sistema para transformar o manipular datos, debe generar los flujos de datos de salida a partir de los de entrada, más una información constante o variable al proceso.



El proceso nunca es el origen ni el final de los datos, puede transformar un flujo de datos de entrada en varios de salida, y es necesario siempre, como intermediario entre entidad externa y almacén de datos.

Todo proceso, independientemente del nivel del diagrama, debe tener al menos una entrada y una salida.

- **Flujo de datos:**

- Representa el movimiento de los datos.
- Muestra la comunicación entre los procesos y los almacenes o entidades externas.
- Un flujo de datos entre dos procesos representa la transferencia lógica de información, sin implicar sincronización temporal ni control de ejecución entre ellos.
- Los flujos de datos entre procesos y almacenes pueden ser de tres tipos:

- » De consulta.

El flujo es desde el almacén al proceso que consulta los datos.

- » De actualización.

Es un Flujo, dónde los datos se alteran, creando, modificando o eliminando los ya existentes. Hay dos tipos de actualizaciones:

- » Actualización completa: Realiza un vaciado completo y una recarga de los datos.
- » Actualización incremental: Procesa un subconjunto de los datos en función de las reglas basadas en el tiempo, expresadas como un filtro, que haya configurado.

- » De diálogo.

Debe haber flujo en ambas direcciones, entre proceso y entidad externa (entrada y salida de información).

DFD orientados al control (modelo extendido)

- **Proceso de control:** (En sistemas orientados al control de datos).

Representa procesos que coordinan y sincronizan las actividades de otros procesos del diagrama de flujo de datos. Es un proceso que no transforma datos sino que decide cuando se ejecuta otro proceso (si debe de activarse, de esperar, de detenerse).

- **Flujo de control:** (En sistemas orientados al control de datos).

Representa el flujo entre un proceso de control y otro proceso.

El flujo de control que sale de un proceso de control activa al proceso que lo recibe y el que entra le informa de la situación de un proceso.



A diferencia de los flujos tradicionales, que pueden considerarse como procesadores de datos porque reflejan el movimiento y transformación de los mismos, los flujos de control no representan datos con valores, sino que, en cierto modo, se trata de eventos que activan los procesos (señales o interrupciones).



+ Info

Muchas veces, se relaciona el almacén con ficheros o bases de datos, pero, un almacén puede ser cualquier soporte de información (documentos en papel, fichas, etcétera).

2.3.2. Representación Gráfica de la Notación

Hay varios tipos de notación y cada uno muestra sus variantes según el autor.

Sin embargo, hay dos tipos de notación que son los más utilizados:

- Yourdon & Coad.
- Gane & Sarson.

Definen los mismos objetos, pero con distintas representaciones visuales.

Elemento	Yourdon & Coad	Gane & Sarson
Entidad		
Proceso		 



Elemento	Yourdon & Coad	Gane & Sarson
Almacén de datos		
Flujo de datos		



El experto opina

Yourdon & Coad es el más utilizado para análisis y diseño de sistemas. (También es parecida, la notación de Yourdon & DeMarco).

Sin embargo, Gane & Sarson posiblemente sea el más utilizado para representar sistemas de información.

2.3.3. Descomposición o explosión en niveles

Técnica top-down

Los diagramas de flujo de datos han de representar el sistema de la forma más clara posible.

Para ello, **su construcción** se basa en el principio de descomposición o explosión en distintos niveles de detalle, que se realiza de arriba abajo (**top-down**), es decir, se empieza en el nivel más general y se termina con el máximo nivel de detalle, pasando por sucesivos niveles intermedios.

La descomposición de cada proceso de un DFD origina otro DFD. Las entradas y salidas (E/S) de un proceso deben ser iguales a las del DFD en que se descompone.

La explosión de cada proceso de un DFD origina otro DFD.

Es imprescindible comprobar que se mantiene la consistencia de información entre ellos: la información de E/S de un proceso cualquiera, se corresponde con la información de E/S del DFD en el que se descompone.



En cualquiera de las explosiones puede aparecer un proceso que no necesite descomposición, se le denomina **Proceso primitivo**.

En un Proceso Primitivo, solo se detalla su entrada y su salida, y una descripción de lo que realiza.

En la construcción, debemos evitar si es posible, la descomposición desigual, es decir, que un nivel necesite ser particionado en uno o varios niveles más y otro nivel contenga un **proceso primitivo**.

Composición

El modelo de procesos deberá contener:

- Diagrama de contexto (nivel 0).

El diagrama de contexto tiene como objetivo delimitar el ámbito del sistema con el mundo exterior definiendo sus interfaces.

- Diagrama 0 (nivel 1).

N diagramas de nivel 2, donde N es el número de procesos del nivel 1.

En función a la complejidad del proceso habrá:

- Los DFD de niveles intermedio que sean necesarios.
- Varios procesos primitivos (DFD en el último nivel de detalle).

Construcción

Aunque el número de niveles depende del sistema y de su tamaño, seguiremos normalmente los siguientes pasos:

1. Se crea el diagrama de contexto, conocido como DFD de Nivel "0".

Diagrama de contexto tiene como objetivo delimitar el ámbito del sistema con el mundo exterior definiendo sus interfaces. Es el de más alto nivel.

Es de gran utilidad para los niveles posteriores de análisis como herramienta de balanceo.

Contendrá:

- Un único proceso que corresponde con el sistema en estudio.
- Un conjunto de entidades externas que representan la procedencia y destino de la información.
- Un conjunto de flujos de datos que representan los caminos por los que fluye dicha información.



2. Se descompone el proceso en otro Diagrama: Nivel 1, subsistemas.

Cada subsistema:

- Contendrá los procesos principales o subsistemas.
- Un subsistema es un conjunto de procesos que colaboran para ofrecer una funcionalidad.

3. Se crean N diagramas de Nivel 2: funciones de cada subsistema.

Donde N es el número de procesos del nivel1.

4. Se crean DFD intermedios: subfunciones asociadas.

Se siguen descomponiendo los procesos hasta llegar a nivel suficiente de detalle.

5. Procesos primitivos.

DFD en el último nivel de detalle, aquel que no se puede descomponer. Se detallará su entrada, su salida y una descripción de lo que realiza. También se denomina función elemental o proceso elemental.

Niveles de DFD

- Contexto.
- Nivel 1: subsistemas.
- Nivel 2: funciones de los subsistemas.
- DFD intermedios (los necesarios).
- Procesos primitivos (último nivel DFD).

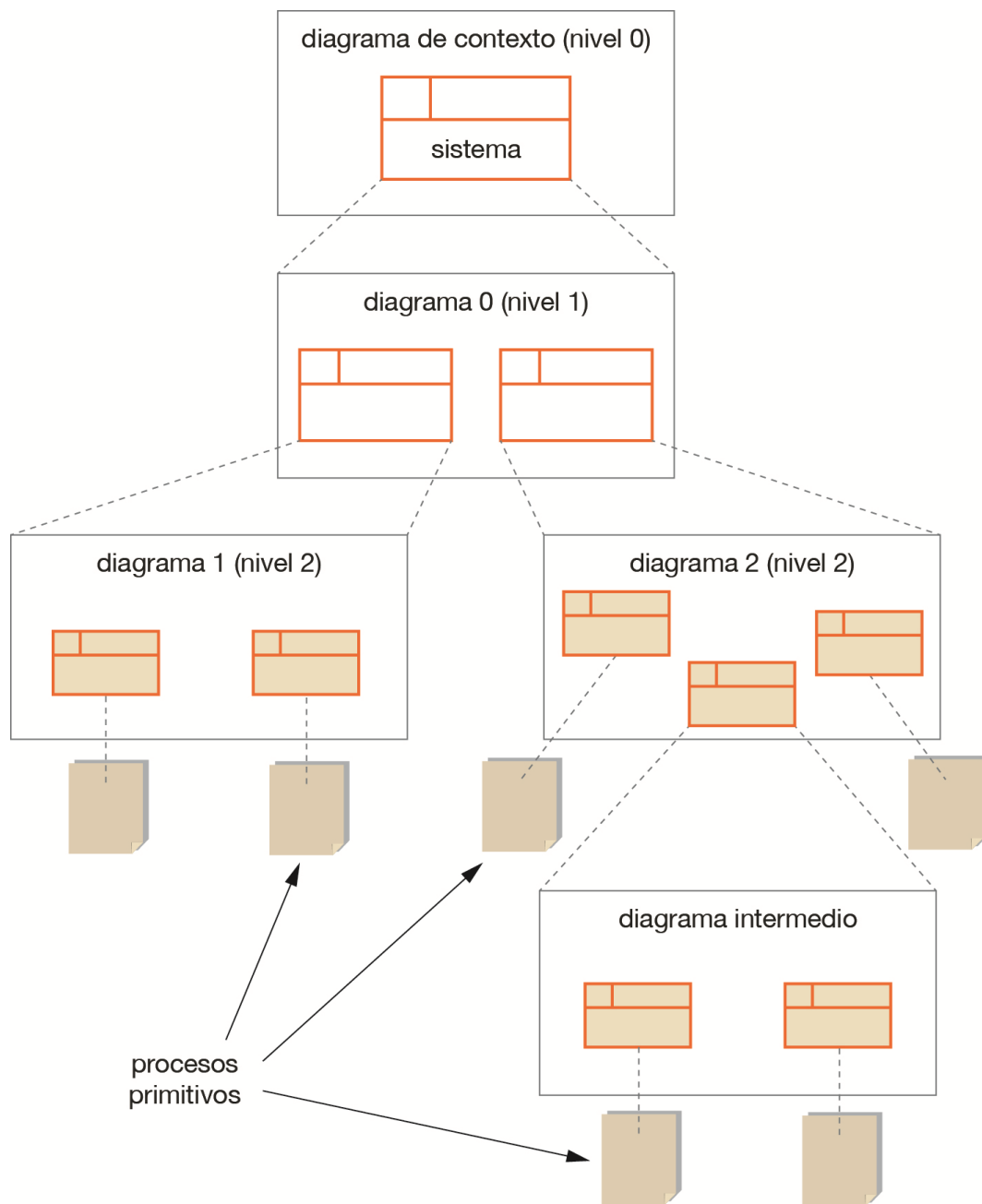


+ Info

El número de niveles depende del sistema y de su tamaño, por lo que no hay que esforzarse demasiado en conseguir esta estructura.



A continuación, se muestra un ejemplo gráfico que representa la de descomposición jerárquica de los diagramas de flujo de datos.

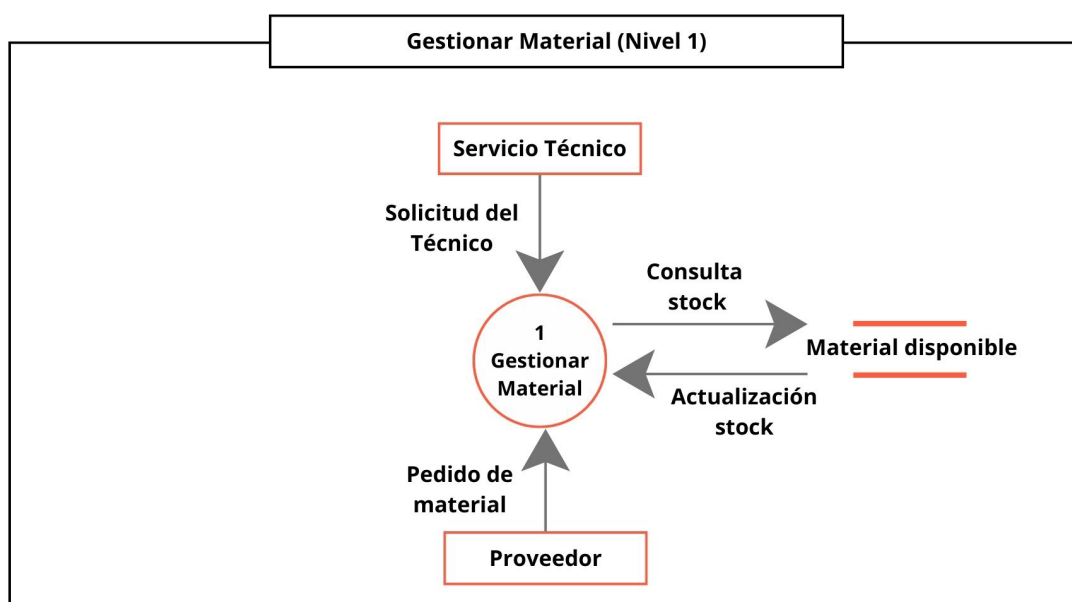
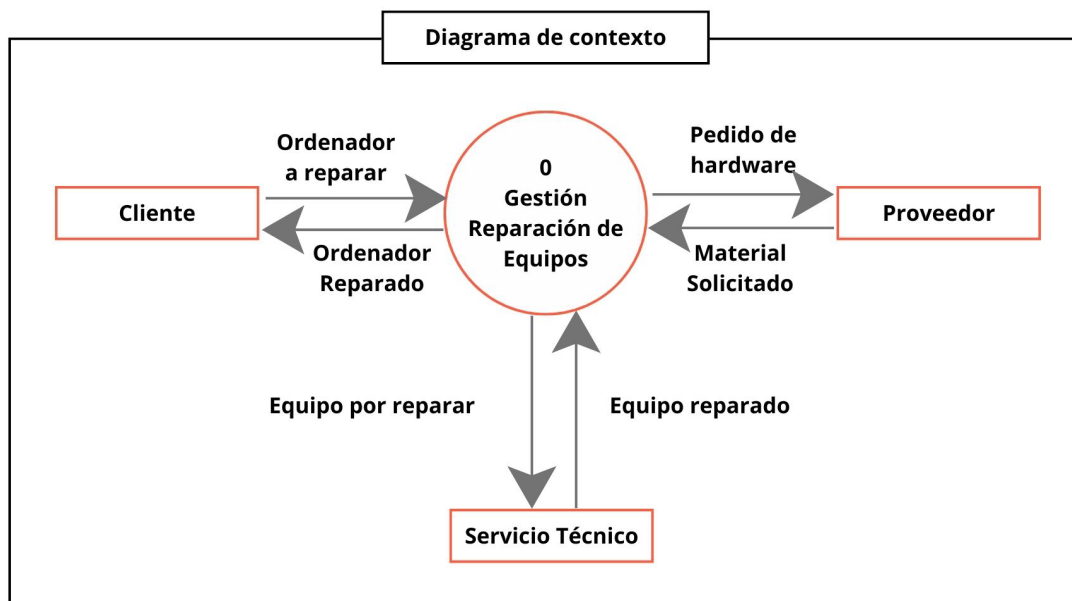


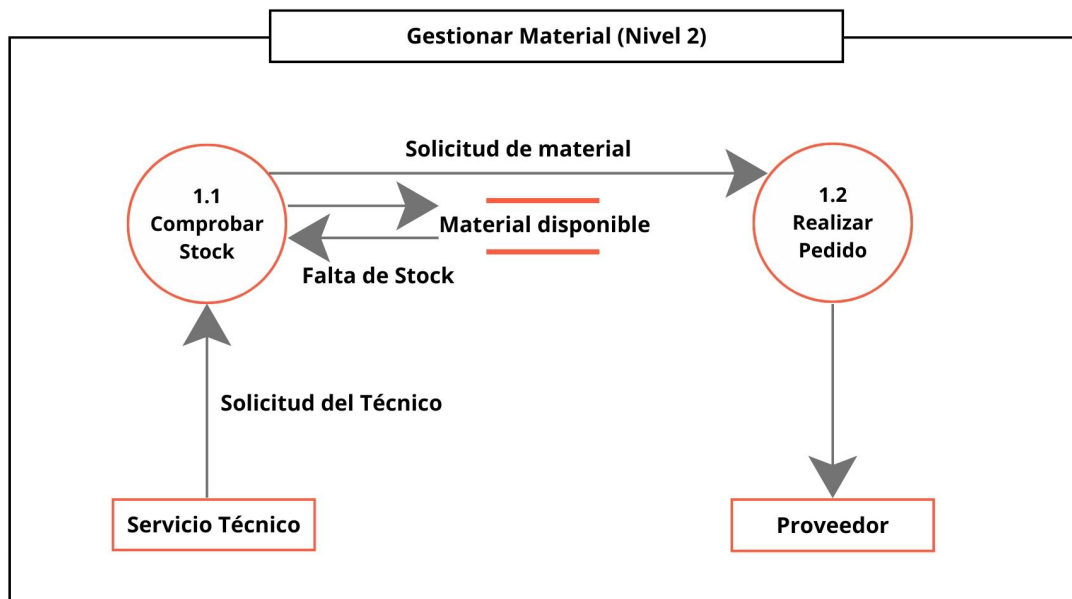


Ejemplo

Vamos a ver un ejemplo, para que veas de forma gráfica la técnica top-down, y comprendas mejor lo estudiado.

Ejemplo de un servicio técnico de informática, teniendo en cuenta sólo averías de hardware:





3. Fase 2. Diseño lógico



Etapa del diseño lógico. Entradas y salidas

En esta fase 2, Diseño lógico, vamos a convertir el esquema conceptual creado en la Fase 1 (Diseño conceptual) en un esquema lógico.

Por tanto, el diseño lógico, parte del volumen de datos con el que se va a trabajar y los criterios de rendimiento.

Las estructuras de datos utilizadas dependerán del modelo utilizado, el cual debe poder ser procesado por el SGBD escogido. Por lo tanto, el diseño lógico depende del modelo de datos del SGBD que se vaya a utilizar, pero no depende de detalles de implementación física.

El modelo lógico es el lenguaje que se utiliza para describir los esquemas.

El modelo más utilizado es el modelo lógico relacional (para relacionales, que son los más utilizados).

El objetivo de la fase de Diseño Lógico, es obtener una representación de las estructuras de datos que se usarán en la base de datos. Estas estructuras dependerán del modelo de datos escogido.



3.1. Flujogramas

Un flujograma (también conocido como diagrama de flujo) es la representación gráfica de un algoritmo o proceso.

Estos diagramas utilizan símbolos con significados definidos que representan los pasos del algoritmo y representan el flujo de ejecución mediante flechas que conectan los puntos de inicio y de fin de proceso.

¿Para qué sirve un flujograma?

Las principales utilidades del flujograma son (Pardo, 2012):

- El proceso se entiende más fácilmente que leyendo un texto, incluso para personas no familiarizadas con él.
- Los agentes involucrados al observar visualmente el proceso pueden llegar más fácilmente a un acuerdo sobre los métodos que hay que seguir.
- Se puede utilizar para mejorar, identificar problemas, establecer recursos, coordinar acciones, delimitar tiempos...
- Deja bien definidas las responsabilidades y funciones de cada uno de los agentes que intervienen.
- Es útil para establecer indicadores operativos.
- Facilita el diseño de nuevos procesos.
- Apoya en la formación personal.
- Permite mejorar la gestión de la organización.

3.1.1. Tipos

Los flujogramas pueden ser de dos tipos:

- **Tipo matricial:** los agentes que intervienen en el proceso aparecen en la cabecera del dibujo y las actividades desempeñadas se encuentran subordinadas a ellos. Se pueden construir de arriba abajo (recomendado) o de izquierda a derecha.
- **Tipo lineal:** las actividades del proceso aparecen una debajo de otra por orden de ejecución.



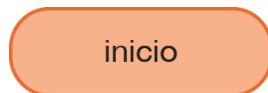
El experto opina

Te aconsejamos utilizar el lineal. Es más fácil de construir.

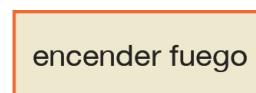
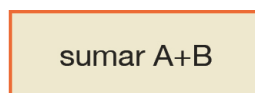
3.1.2. Elementos y su notación

Hay múltiples signos que se utilizan para la generación de flujogramas. Sin embargo, hay algunos básicos que son los más utilizados:

- **Inicio/fin:**
 - Indica el inicio y el final del diagrama de flujo.
 - Está reservado a la primera y última actividad.
 - Un proceso puede tener varios inicios y finales.



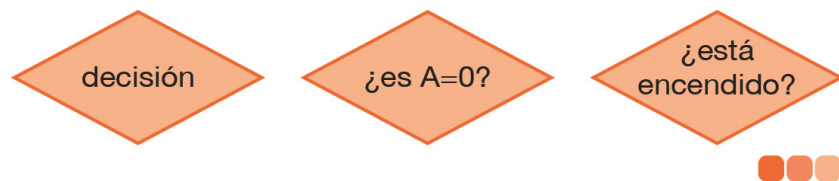
- **Actividad o tarea:**
 - El nombre debe incluir siempre un verbo de acción (aunque a veces se utiliza una operación matemática).
 - Las cajas se pueden numerar.





• Decisión:

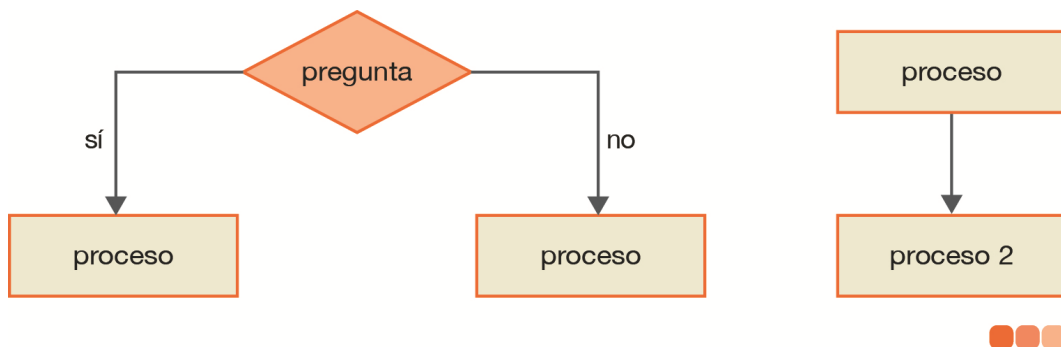
- Según el valor de la respuesta, tomará un camino u otro.
- El nombre debe ser una pregunta.



• Flujo:

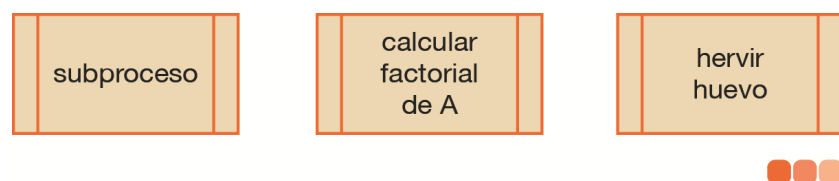
- Se representa como una flecha **con una única dirección**.
- Indica la dirección en que se van ejecutando los distintos elementos del proceso.
- Une símbolos entre sí.
- Pueden tener una etiqueta.

Por ejemplo, las salidas de una decisión pueden tener "sí" y "no" para indicar la salida cuando la respuesta es sí y la salida cuando es no.



• Subproceso:

- Es un proceso que se ejecuta dentro del proceso principal.
- Se podrá desarrollar con otro diagrama de flujo.





- **Entradas y salidas:**

- Representan entradas que se utilizan en las actividades y salidas generadas por el proceso.
- Existen muchos tipos.

Algunos ya no se utilizan (como la lectura de tarjetas perforadas). Vamos a ver los más interesantes.



- **Referencias o conectores:**

- Indican que el flujo continúa por otro lugar.
- Pueden conectar dos lugares de la misma página o de distintas páginas.



conector a otra página



conector dentro de la misma página



+ Info

No tienen por qué tener color, aunque estos hacen que sea más agradable y más sencillo de visualizar.



3.1.3. Reglas de construcción

Existen un conjunto de reglas básicas para la construcción de un diagrama de flujo.

- **Regla 1.**

Todo diagrama de flujo debe tener un inicio y un final.

- **Regla 2.**

Las líneas utilizadas para indicar la dirección del flujo del diagrama deben ser rectas: verticales u horizontales.

- **Regla 3.**

Todas las líneas utilizadas para indicar la dirección del flujo del diagrama deben estar conectadas. La conexión puede ser a un símbolo que exprese lectura, proceso, decisión, impresión, conexión o fin del diagrama.

- **Regla 4.**

Los diagramas de flujos deben construirse de arriba hacia abajo (*top-down*) y de izquierda a derecha (*left to right*).

- **Regla 5.**

La notación utilizada en el diagrama de flujo no debe depender del lenguaje de programación. La solución presentada se puede escribir después en varios lenguajes de programación.

- **Regla 6.**

Al realizar una tarea compleja, es conveniente escribir comentarios que expresen o ayuden a entender lo que hayamos hecho.

- **Regla 7.**

Si la construcción del diagrama de flujo ocupara más de una hoja, utilizaríamos los conectores adecuados y enumeraríamos las páginas correspondientes.

- **Regla 8.**

A un símbolo determinado pueden llegar varias líneas de flujo (convergencia), pero de un símbolo de proceso solo puede salir una sola línea. Si se requiere más de una ruta de salida, se debe utilizar obligatoriamente un símbolo de decisión (rombo).

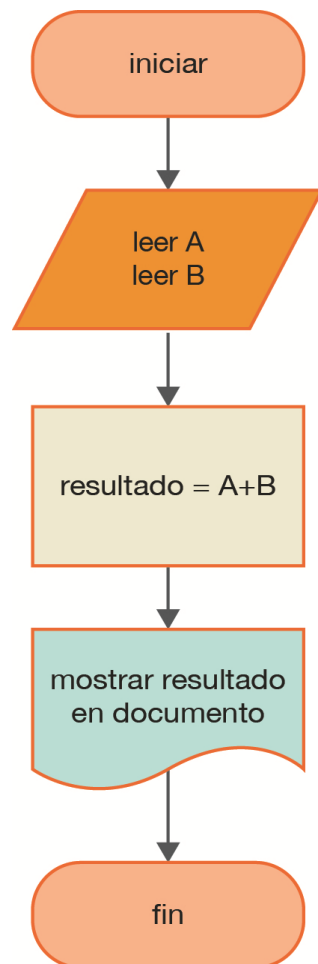


3.1.4. Ejemplos de Flujogramas

Ahora que has aprendido conceptos, normas y reglas para realizar un análisis de datos y procesos, vas a ver dos ejemplos.

Ejemplo: "Suma de dos números"

Vamos a leer dos números por teclado, los vamos a sumar y mostraremos el resultado en un documento.



Suma de dos números

Un programador o analista, concluye que tiene que obtener 2 datos A y B, que (podrían estar almacenados o ser introducidos por pantalla), que hay que realizar un proceso de suma, y después mostrar el resultado.



3.1.5. Pensar como un programador

Es hora de que pienses como un programador. Veamos qué tal se te da con el próximo ejemplo.



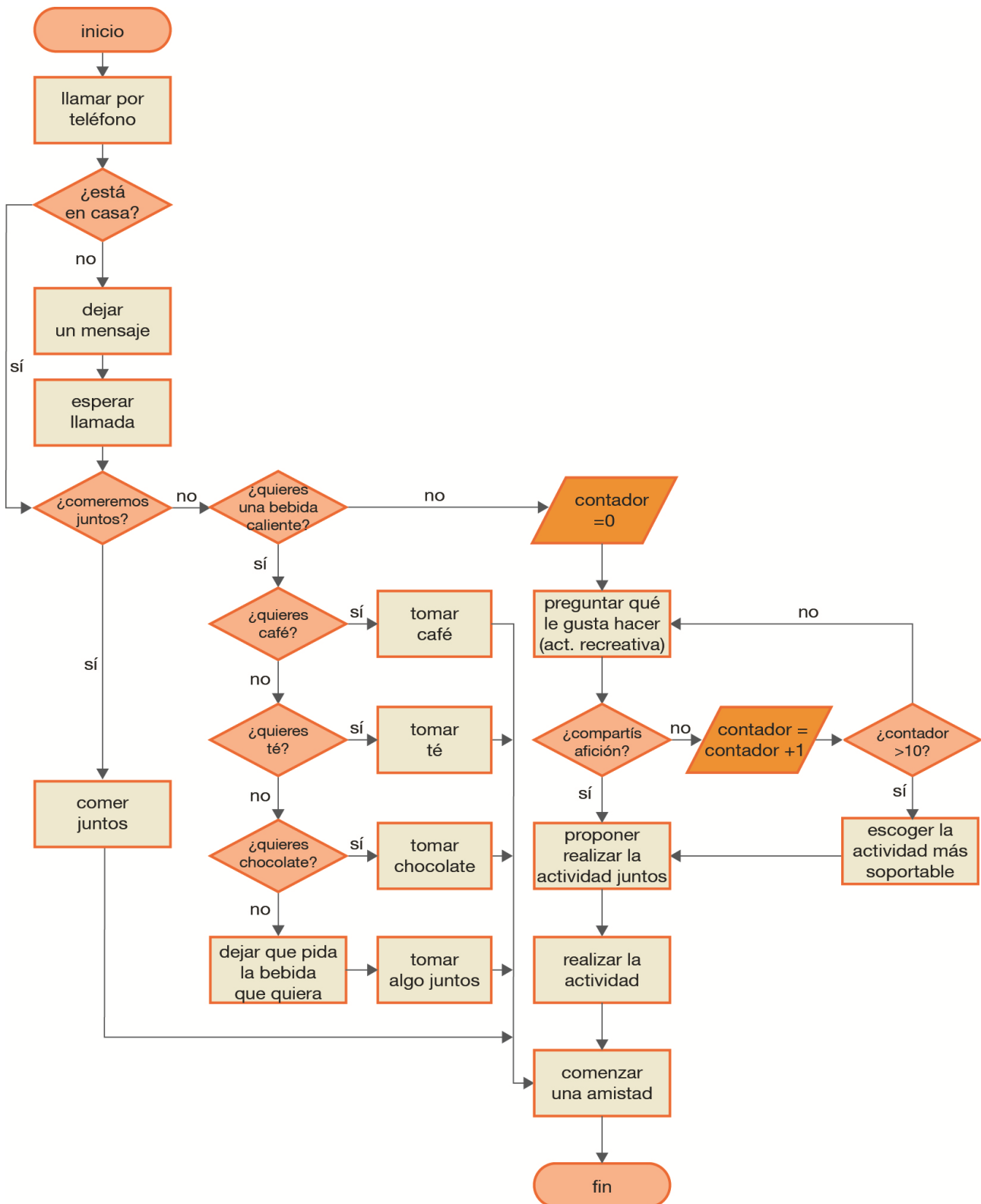
Fuente: Public domain vectors

Para comprobar todo lo que has aprendido, vamos a ver un algoritmo de para quedar con un amigo/a.



Reto

Hay una posible **situación, que no está reflejada**, intenta saber cuál es antes de continuar al siguiente apartado. Ahora tienes que utilizar la lógica de ser programador. Y también te sirve tu experiencia a la hora de quedar con amigos o amigas...





Solución



Atención

¿Has detectado algún error?

Este diagrama da por supuesto, que la persona a la que has llamado, si no está en casa, va a devolverte la llamada.

¿Pero... y si no te llama?

¿La volverías a llamar?

Cuántas veces volverías a llamar si no está en casa y no te devuelve la llamada...



Importante

Ahora ya empiezas a pensar cómo un programador.

Adelantándote a las necesidades que el usuario aún no sabe que tiene o que podrá tener en el futuro.

3.2. Información de la carga y criterios de rendimiento

Es importante conocer la siguiente información (requisitos de software) sobre las bases de datos para el correcto diseño del esquema lógico:

- El volumen de datos con el que vamos a trabajar.
- Las consultas y transacciones que se van a realizar y la frecuencia de estas.



Otro factor que se debe tener en cuenta son los criterios de rendimiento acordados con el cliente. Algunos de ellos son:

- Tiempo de respuesta medio y máximo.
- Espacio de almacenamiento que ocupará la base de datos.
- Utilización de CPU.

3.3. El modelo lógico relacional (B.D. Relacional)

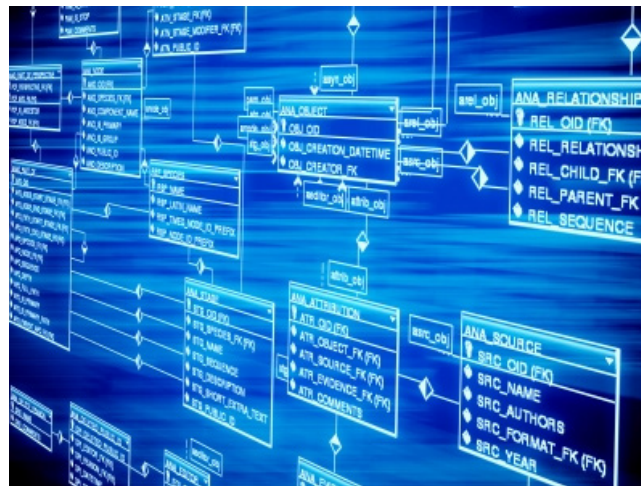


Imagen cortesía de David Castillo Dominici de <http://www.freedigitalphotos.net>

El modelo de datos relacional organiza y representa los datos en forma de tablas o relaciones.



Entradas y salidas del diseño lógico

Los usuarios perciben una B.D. relacional, como un conjunto de tablas.

En el modelo relacional, la base fundamental, es el uso de relaciones. Se utiliza un grupo de tablas para representar datos y las relaciones entre ellos.

A cada tabla se le asigna un nombre exclusivo.



3.3.1. Conceptos básicos

Para entender bien la estructura de una base de datos relacional, debemos saber unos conceptos fundamentales:

- **Tabla.**

Objeto que contiene una colección de datos para un tema específico. Las tablas constan de filas y columnas.

- **Columna.**

Componente vertical de una tabla de base de datos. Una columna tiene un nombre y un tipo de datos específico, por ejemplo, carácter, decimal o entero.

- **Fila.**

Componente horizontal de una tabla, que consta de una secuencia de valores, uno para cada columna de la tabla.

- **Vista.**

Tabla lógica que se basa en datos almacenados en varias tablas (utiliza una sentencia SELECT).

- **Tupla.**

Es una fila de la tabla. También se denomina instancia o registro.

- **Cardinalidad.**

El número de tuplas de una tabla.

- **Atributo.**

Es una columna de la tabla (equivale a un campo de una tupla).

- **Grado.**

El número de atributos se llama grado.

- **Dominio.**

Es una colección de valores, de los cuales uno o más atributos obtienen sus valores reales.



- **Clave (o llave) candidata.**

Es un atributo (o conjunto de atributos) de una relación R que cumple las siguientes propiedades:

- Unicidad.
- Minimalidad.

- **Clave (o llave) primaria.**

Si una relación (tabla) tiene más de una clave candidata, se escoge una de ellas como primaria y el resto pasan a ser claves alternativas.

- **Clave foránea (ajena o extranjera).**

Es un atributo (o conjunto de atributos) de una relación (tabla) que es clave primaria de otra relación (tabla).

Se utiliza para referenciar a la tupla de la otra relación cuya clave primaria coincida con el valor de la clave foránea.

- **Reglas de Integridad.**

Son un conjunto de reglas que debe cumplir la Base de Datos.

Dependencia funcional

Una dependencia funcional es una relación entre uno o más atributos.

Por ejemplo, si se conoce el valor de DNI (Documento Nacional de Identidad-España), se determinan unívocamente los atributos Nombre y Apellidos.

Las dependencias funcionales del sistema se escriben utilizando una flecha ($\rightarrow$), de la siguiente manera:

$\text{DNI} \rightarrow \text{Nombre, Apellidos}$

B es funcionalmente dependiente de A, si A determina el valor de B ($A \rightarrow B$)



Ejemplo

Los primeros números del código postal nos indican la población, por lo que:

codigoPostal $\rightarrow$ población



Características de la dependencia funcional:

- La dependencia funcional es una noción semántica.
- Cada dependencia funcional es una clase especial de regla de integridad.
- Cada dependencia funcional representa una relación $A \rightarrow B$, siendo A y B atributos sencillos o grupos de atributos (DNI $\rightarrow$ Nombre, Apellido o Nombre, Apellido, Dirección $\rightarrow$ telefono, año de nacimiento). La dependencia funcional garantiza que a A siempre le corresponda B.
- Son propiedades inherentes al contenido semántico de los datos, que se han de cumplir para cualquier extensión del esquema de relación.
- Se tratan de restricciones de integridad que permiten conocer qué interrelaciones existen entre los atributos del mundo real.
- Son invariantes en el tiempo.

3.3.2. Reglas de Integridad en el Modelo Relacional

Una base de datos, debe tener exactitud en la información que contiene, asegurar la seguridad de los datos, y permitir el acceso a múltiples usuarios a dichos datos en tiempos paralelos.

Existen varias reglas que debe cumplir una base de datos en el modelo relacional:

- Regla de integridad de entidad de la clave primaria.
- Regla de integridad de unicidad de la clave primaria.
- Regla de integridad referencial.
- Regla de Datos-Atributos requeridos.
- Regla de integridad de dominio.
- Reglas de negocio.

Estudiamos cada una de ellas a continuación.

3.3.2.1. Regla de integridad de entidad de la clave primaria

Integridad de entidades. El identificador de una entidad (claves primarias) no admiten valores nulos.



Esta regla, dispone que los atributos de la clave primaria de una relación no pueden tener valores nulos. Esta regla es necesaria para que los valores de las claves primarias puedan identificar las tuplas individuales de las relaciones. Si las claves primarias tuviesen valores nulos, es posible que algunas tuplas no se pudieran distinguir.

Un SGBD relacional tendrá que garantizar el cumplimiento de esta regla de integridad en todas las inserciones y en todas las modificaciones que afecten a atributos que pertenecen a la clave primaria de la relación.

3.3.2.2. Regla de integridad de unicidad de la clave primaria

Está relacionada con la definición de clave primaria, que establece que toda clave primaria que se elija para una relación no debe tener valores repetidos.

3.3.2.3. Regla de integridad referencial

No deben existir valores de clave foránea sin concordancia.

La necesidad de esta regla es debido a que las claves foráneas tienen por objetivo establecer una conexión con la clave primaria que referencian.

Esta regla está relacionada con la clave foránea. Los valores de claves foráneas deben existir en la clave primaria referenciada o bien deben ser valores nulos.

Una clave foránea enlaza cada tupla de la relación hijo con la tupla de la relación padre que tiene el mismo valor en su clave primaria.

La integridad referencial dice que si una clave foránea tiene un valor (si es no nula), ese valor debe ser uno de los valores de la clave primaria a la que referencia.

Hay que controlar muy bien la clave foránea, en caso de borrado o modificación (de los datos hijo). El fin principal de una restricción de clave externa es controlar los datos que pueden almacenarse en la tabla de la clave externa; también controla los cambios realizados en los datos de la tabla de la clave principal.

Ejemplo: si tenemos una tabla "Tdatos-ventas-vendedor", con una clave foránea (cod_departamento+cod_vendedor), no podemos borrar o modificar las claves principales cod:departamento ni cod_vendedor, sin comprobar cómo afectará a la tabla "Tdatos-ventas-vendedor", no habrá relación.

Con una restricción en la clave foránea se evita esta situación, garantizando que no se puedan realizar cambios en los datos de la tabla de la clave principal, si esos cambios anulan el vínculo con los datos de la tabla de la clave foránea.



Si se intenta eliminar la fila de una tabla de la clave principal o cambiar un valor de clave principal, la acción no progresará si el valor de la clave principal cambiado o eliminado corresponde a un valor de la restricción de clave externa de otra tabla.

Hay varios aspectos a tener en cuenta sobre las claves foráneas para lograr que se cumpla la integridad referencial.

Supuestos

Pregunta 1:

¿Admite nulos la clave foránea?

Admisión de valor nulo en la clave foránea: Cada clave foránea expresa una relación. Si la participación de la entidad hijo en la relación es total, entonces la clave foránea no admite nulos; si es parcial, la clave foránea debe aceptar nulos.

Pregunta 2:

¿Qué hacer cuando se quiere borrar una ocurrencia de la entidad padre que tiene algún hijo? (se quiere borrar una tupla que está siendo referenciada por otra tupla a través de una clave foránea)

Posibilidades:

- Restringir: evitar que se puedan borrar tuplas que están siendo referenciadas por otras tuplas.
- Propagar: Se borra la tupla deseada y se propaga el borrado a todas las tuplas que le hacen referencia.
- Anular: Se borra la tupla deseada y todas las referencias que tenía se ponen, automáticamente, a nulo (si la clave foránea acepta nulos).
- Valor por defecto: Se borra la tupla deseada y todas las referencias toman, automáticamente, el valor por defecto (si se ha especificado un valor por defecto para la clave foránea).

Pregunta 3:

¿Qué hacer cuando se quiere modificar la clave primaria de una tupla que está siendo referenciada por otra tupla a través de una clave foránea?

Se actúa de la misma forma que en la pregunta 2.



Formas de mantener la integridad referencial:

Podemos utilizar diferentes métodos para mantener la integridad referencial. Son los siguientes:

- **Restricción:**

Garantizar, que no se puedan realizar cambios en los datos de la tabla de la clave principal, si esos cambios anulan el vínculo con los datos de la tabla de la clave foránea.

- La restricción en caso de modificación, consiste en no permitir modificar ningún atributo de la clave primaria de una tupla si tiene una clave primaria referenciada por alguna clave foránea.
- La restricción en caso de borrado, consiste en no permitir borrar una tupla si tiene una clave primaria referenciada por alguna clave foránea.

- **Actualización en cascada:**

Consiste en permitir la operación de actualización de la tupla, y en efectuar operaciones compensatorias que propaguen en cascada la actualización a las tuplas que la referenciaban.

La actualización en cascada en caso de borrado consiste en permitir el borrado de una tupla T que tiene una clave primaria referenciada, y borrar también todas las tuplas que referencian T y la actualización en cascada en caso de modificación consiste en permitir la modificación de atributos de la clave primaria de una tupla T que tiene una clave primaria referenciada, y modificar del mismo modo todas las tuplas que referencian T.

- **Anulación:**

Consiste en permitir la operación de actualización de la tupla y en efectuar operaciones compensatorias que pongan valores nulos a los atributos de la clave foránea de las tuplas que la referencian; esta acción se lleva a cabo para mantener la integridad referencial.

Los SGBD relacionales permiten establecer que un determinado atributo de una relación no admite valores nulos, sólo se puede aplicar la política de anulación, si los atributos de la clave foránea sí los admiten.

Más concretamente, la anulación en caso de borrado, consiste, en permitir el borrado de una tupla T que tiene una clave referenciada, y, además, modificar todas las tuplas que referencian T, de modo que los atributos de la clave foránea correspondiente tomen valores nulos.

Y la anulación en caso de modificación, consiste en, permitir la modificación de atributos de la clave primaria de una tupla T, que tiene una clave referenciada, y, además, modificar todas las tuplas que referencian T, de modo que los atributos de la clave foránea correspondiente tomen valores nulos.



3.3.2.4. Regla de Datos-Atributos requeridos

Algunos atributos deben contener valores en todo momento, es decir, no admiten valores nulos.

3.3.2.5. Regla de integridad de dominio

Los atributos deben tomar valores dentro de su dominio que se le ha definido.

Está relacionada con la noción de dominio. Esta regla establece dos condiciones.

- **Primera condición.**

Consiste en que un valor no nulo de un atributo A_x debe pertenecer al dominio del atributo A_x ; es decir, debe pertenecer a $\text{dominio}(A_x)$. Esta condición implica que todos los valores no nulos que contiene la base de datos para un determinado atributo deben ser del dominio declarado para dicho atributo.

- **Segunda condición.**

Sirve para establecer que los operadores que pueden aplicarse sobre los valores dependen de los dominios de estos valores; es decir, un operador determinado sólo se puede aplicar sobre valores que tengan dominios que le sean adecuados.

3.3.2.6. Reglas de negocio

Cualquier operación que se realice sobre los datos debe cumplir las restricciones que impone el funcionamiento de la empresa.

3.3.3. Metodología de diseño lógico en el modelo relacional



Técnicas de estudio

Puede resultarte útil, buscar un ejemplo simple de esquema conceptual (dos o tres entidades) y que sobre él, vayas aplicando los pasos que vas a estudiar a continuación.



Vamos a diferenciar 2 partes, cada una de las cuales tendrá varios pasos o tareas:

- **1ª parte: Construcción y validación de los esquemas lógicos locales** (para cada vista de usuario):
 1. Convertir los esquemas conceptuales locales en esquemas lógicos locales.
 2. Derivar un conjunto de relaciones (tablas) para cada esquema lógico local.
 3. Validar cada esquema mediante la normalización.
 4. Validar cada esquema frente a las transacciones del usuario.
 5. Documentar el esquema relacional (tablas, claves, restricciones).
 6. Definir las restricciones de integridad.
 7. Revisar cada esquema lógico local con el usuario correspondiente.

La 2ª parte no comenzará hasta terminar la 1ª parte

- **2ª parte: Construcción y validación del esquema lógico global:**
 1. Integrar los esquemas lógicos locales en un esquema lógico global, resolviendo conflictos de nombres, dominios y estructuras.
 2. Validar el esquema lógico global.
 3. Estudiar el crecimiento futuro.
 4. Documentar/Revisar el esquema relacional global final.
 5. Revisar el esquema lógico global con los usuarios.

1) Convertir los esquemas conceptuales locales en esquemas lógicos locales

En este paso, transformamos de cada esquema conceptual las estructuras que no se representan directamente en el modelo relacional:

- **Relaciones N:N.** Sustituimos cada una de ellas por una nueva entidad intermedia y dos nuevas relaciones 1:N entre esta entidad y las entidades originales.

La nueva entidad es una entidad asociativa, cuya clave primaria se construye a partir de las claves primarias de las entidades originales.



- **Relaciones entre tres o más entidades.** Sustituimos cada relación por una entidad asociativa, que se relaciona con cada una de las entidades originales. La cardinalidad de estas nuevas relaciones binarias dependerá de cada caso.
- **Relaciones recursivas** (se transforman mediante una clave foránea que referencia a la misma tabla).
- **Relaciones con atributos.**
- **Atributos multivaluados.**
- **Relaciones uno a uno.** Hay que revisar las relaciones uno a uno porque normalmente las dos entidades relacionadas podrían combinarse en una sola entidad.
- **Relaciones redundantes.** Una relación es redundante cuando se puede obtener la misma información que ella aporta mediante otras relaciones.

2) Derivar un conjunto de relaciones (tablas) para cada esquema lógico local

En este paso, obtenemos un conjunto de relaciones (tablas) para cada uno de los esquemas lógicos locales.

Representamos las entidades y relaciones entre entidades descritas en cada vista de usuario. Cada relación de la base de datos se identificará mediante un nombre seguido de sus atributos entre paréntesis. El atributo o atributos que forman la clave primaria se subrayan.

A continuación, vamos a ver cómo se transforman los elementos que aparecen en los esquemas lógicos (entidades y relaciones) en relaciones del modelo relacional (tablas).

- **Entidades fuertes.**
 - Crear una relación para cada entidad fuerte que incluya todos sus atributos simples.
 - De los atributos compuestos incluir solo sus componentes.
 - Cada uno de los identificadores de la entidad será una clave candidata.
 - De entre las claves candidatas hay que escoger la clave primaria y el resto serán claves alternativas.
- **Entidades débiles.**
 - Crear una relación para cada entidad débil incluyendo todos sus atributos simples.
 - De los atributos compuestos incluir solamente sus componentes.



- Añadir una clave foránea a la entidad de la que depende.
- Determinar la clave primaria de la nueva relación.
- **Relaciones binarias 1:1.**
 - El ejemplo paradigmático es aquel en el que la entidad hijo participa de forma total (obligatoria) en la relación, mientras que la entidad padre participa de forma parcial (opcional). En este caso, los atributos de la clave primaria de la entidad padre se incluyen como clave foránea en la tabla que representa a la entidad hijo.
 - Puede darse el caso de que ambas entidades participen bien sea de forma parcial o total en la relación.
 - » Si una de las entidades no participa en ninguna otra relación, se integran las dos entidades en una sola relación-entidad (tabla).
 - » La elección de padre e hijo es arbitraria.
- **Relaciones binarias 1:N.**
 - Entidad padre. Es la entidad en la que cada elemento se relaciona con N elementos de la otra entidad (un padre tiene N hijos).
 - Entidad hija. Es la entidad en la que cada elemento se relaciona con un solo elemento de la otra entidad (un hijo tiene un padre).
 - Se incluyen los atributos de la clave primaria de la entidad padre como clave foránea en la relación (tabla) que representa a la entidad hijo (igual que con las 1:1).
- **Jerarquías de generalización.**

Entidad padre. Es la entidad genérica.

Entidad hijo. Son las entidades especializadas.

 - **Método 1:**
 - » Cualquier tipo de jerarquía.
 - » Crear una relación por cada entidad.
 - » Las relaciones de las entidades hijo heredan como clave primaria la de la entidad padre.
 - » Por lo tanto, la clave primaria de las entidades hijo es también una clave foránea al padre.



- **Método 2:**
 - » Cualquier tipo de jerarquías.
 - » Integrar todas las entidades en una relación.
 - » Incluir en la relación los atributos de la entidad padre, los atributos de todos los hijos y un atributo discriminativo para indicar el caso al cual pertenece la entidad en consideración.
 - » Si la jerarquía es superpuesta, el atributo discriminativo será multivaluado.
- **Método 3:**
 - » Solo para Jerarquías totales exclusivas.
 - » Crear una relación por cada entidad hijo que heredarán los atributos de la entidad padre.



+ Info

Un atributo multivaluado es aquel que puede tener más de un valor para una instancia. Por ejemplo, si tenemos la tabla “coche” y el atributo “color”, algunos coches tendrán un único color, pero también puede haber un coche con dos o más colores.

3) Validar cada esquema mediante la normalización

Una base de datos no está orientada a ser lo más eficiente posible. El objetivo ahora es conseguir una base de datos normalizada por las siguientes razones:

- Un esquema normalizado organiza los datos de acuerdo con sus dependencias funcionales, es decir, basado en sus relaciones lógicas.
- Un esquema normalizado es robusto y carece de redundancias.
- La normalización produce bases de datos con esquemas flexibles que pueden extenderse con facilidad.



+ Info

La mayoría de los autores marcan como objetivo llegar a la forma normal de Boyce-Codd, para lo cual antes debe estar en primera, segunda y tercera forma normal.

Como veremos, también hay una cuarta y quinta forma normal que tienen una aplicación mucho menor.

4) Validar cada esquema frente a las transacciones del usuario

Esto se debe hacer para garantizar que puede hacer lo que el usuario requiere. Las transacciones del usuario están en la especificación de requisitos. Debe ser capaz de realizar todas las transacciones.

5) Documentar el esquema relacional (tablas, claves, restricciones)

Se plasma el esquema lógico local como conjunto de tablas con sus claves y restricciones, reflejando la visión de datos del usuario y validado previamente por normalización y transacciones.

6) Definir las restricciones de integridad

Las restricciones de integridad son reglas que se imponen para proteger la base de datos y evitar un estado de inconsistencia.

Las Reglas de Integridad, se estudiarán más adelante en mayor profundidad.

7) Revisar cada esquema lógico local con el usuario correspondiente

Para garantizar que cada esquema lógico local es una fiel representación de la vista del usuario, lo mejor es comprobar el esquema y la documentación con el usuario para ver si es correcto y está completo.



2ª parte: Construcción y validación del esquema lógico global.

1) Integrar los esquemas lógicos locales en un esquema lógico global, resolviendo conflictos de nombres, dominios y estructuras

Integramos todos los esquemas locales en un solo esquema global. En un sistema pequeño es sencillo, pero si el sistema es grande debemos seguir una serie de pasos:

1. Revisar los nombres de las entidades y sus identificadores (claves candidatas y clave primaria).
2. Revisar los nombres de las relaciones.
3. Fusionar las entidades comunes de las vistas locales.
4. Incluir sin cambios las entidades propias de una única vista.
5. Fusionar las relaciones comunes de las vistas locales.
6. Incluir sin cambios las relaciones propias de una única vista.
7. Comprobar que no se ha omitido ninguna entidad ni relación.
8. Verificar claves foráneas y consistencia de dominios.
9. Verificar restricciones de integridad.
10. Documentar el esquema relacional global (tablas, claves, restricciones).
11. Actualizar la documentación del diseño lógico global.

2) Validar el esquema lógico global

Lo validamos mediante la normalización y probándolo con las transacciones de los usuarios.

Esto solo se hará con las relaciones que hayan sufrido algún cambio al integrar los esquemas lógicos locales; las relaciones sin modificaciones conservan la validez ya obtenida.

3) Estudiar el crecimiento futuro

Comprobamos que el esquema obtenido puede acomodar los futuros cambios en los requisitos con un impacto mínimo.



4) Documentar/Revisar el esquema relacional global final

Se actualiza la documentación con el esquema relacional definitivo (tablas, claves, restricciones) y el diccionario de datos correspondiente.

5) Revisar el esquema lógico global con los usuarios

Presentamos la documentación y el esquema relacional final a los usuarios para verificar que cubre todos sus requisitos antes de pasar a la implementación.

3.4. Normalización

La normalización de bases de datos es un proceso que consiste en designar y aplicar una serie de reglas a las relaciones obtenidas tras el paso del modelo entidad-relación al modelo relacional. con objeto de minimizar la redundancia de datos, facilitando su gestión posterior.

La normalización es el proceso de organizar los datos de una base de datos.

Tiene como **objetivo** optimizar su diseño, intentando evitar problemas de inconsistencia y redundancia (repetición) de datos.

Si una base de datos no está normalizada, las principales consecuencias son:

- Errores de inconsistencia.
- Incoherencias.
- Actualización.
- Redundancia de datos.

Los datos redundantes desperdician el espacio de disco y crean problemas de mantenimiento (ya que, al hacer un cambio, hay que asegurarse de hacer el cambio en todas las ubicaciones donde se encuentre).

Con la normalización convertimos una tabla con datos complejos en una o más tablas simples que son más fáciles de utilizar.

Es mejor invertir esfuerzo en la fase de diseño de la base de datos que gastarlo en el desarrollo del programa para paliar defectos del diseño de la base de datos. Además, facilita el mantenimiento.



Existen varias reglas para la normalización de bases de datos. Cada regla se denomina una "forma normal". Las más importantes son las tres primeras (definidas por Codd).



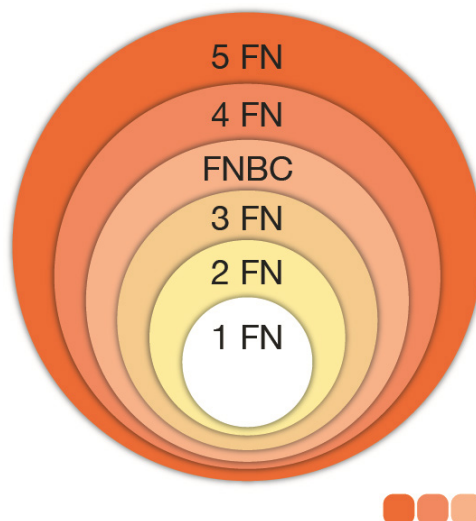
+ Info

Se dice que una base de datos está normalizada si cumple al menos las tres primeras formas normales.

En la mayoría de las ocasiones no se llega más allá.

Si se cumple la **primera regla**, se dice que la base de datos está en la "**Primera Forma Normal**". Si se cumplen **las tres primeras reglas**, la base de datos se considera que está en la "**tercera forma normal**" o que está "**Normalizada**".

3.4.1. Formas normales



Formas normales

La normalización requiere tablas adicionales y puede afectar al rendimiento. Si decides infringir una de las tres primeras reglas de la normalización, asegúrate de tener previstos los problemas que puedan aparecer, como la existencia de datos redundantes y las dependencias incoherentes.



Importante

En los ejemplos, añadiremos a los nombres de atributos el sufijo "PK" si son claves primarias (primary key) y "FK" en caso de ser claves foráneas o ajenas (foreign key).

3.4.1.1. 1Fn

Una tabla se encuentra en primera forma normal si y solo si todos sus atributos contienen valores atómicos (sin grupos repetitivos ni valores multivaluados). Normalmente se define una clave primaria para identificar unívocamente las tuplas.

Ejemplo 1FN

Tabla tbAlumnos:

idAlumnoPK (clave primaria)	nombreAlumno	telefonos
1432	Rafael González Martínez	976111222, 678123456
38	María López Pérez	976999888, 687987654
38	María López Pérez	976999888, 687987654

En esta tabla podemos observar que no se cumple la primera forma normal por tres motivos:

- El atributo "nombreAlumno" no es indivisible, ya que podemos separarlo en tres más pequeños ("nombreAlumno", "apellidoPaternoAlumno", "apellidoMaternoAlumno").
- El atributo "Teléfono" no tiene un valor único, sino que presenta varios valores para una sola tupla.
- La segunda y tercera tupla están repetidas.

Ejemplo 1FN - Solución 1

Para que cumpla la primera forma normal, debemos realizar los siguientes cambios:

- Dividir el atributo "nombreAlumno" en tres partes.
- Dividir el atributo "Teléfono" en dos para poder guardar de forma separada los números de teléfono.
- Eliminar la tupla 3.



Nos quedaría la siguiente tabla:

IdAlumnoPK (clave primaria)	nombre Alumno	apellido PaternoAlumno	apellido MaternoAlumno	telefono1	telefono2
1432	Rafael	González	Martínez	976111222	678123456
38	María	López	Pérez	976999888	687987654

Sin embargo, esta tabla no es del todo óptima. Un alumno podría tener un solo teléfono, lo que haría que el atributo "teléfono2" tuviese un valor nulo, desperdiciando espacio. Por otro lado, un alumno podría tener más de dos teléfonos (fijo, móvil y del trabajo), teniendo que omitir uno de ellos.

Se propone otra solución optimizada:

Ejemplo 1FN - Solución 2

- Dividir el atributo "nombreAlumno" en tres partes.
- Crear la tabla tbAlumnosTelefonos y mover los teléfonos a dicha tabla. Esta tabla tendrá los atributos "IdAlumnoPK" y "telefonoPK" (ambos formarán la clave primaria).
- Eliminar la tupla 3.

Obtendríamos las siguientes tablas:

tbAlumnos:

IdAlumnoPK (clave primaria)	nombre Alumno	apellido PaternoAlumno	apellido MaternoAlumno
1432	Rafael	González	Martínez
38	María	López	Pérez

tbAlumnosTelefonos:

IdAlumnoPK (clave primaria)	telefonoPK (clave primaria)
1432	976111222
1432	678123456
38	976999888
38	687987654



De esta forma, un alumno podría tener un teléfono sin malgastar espacio y más de dos si fuera necesario.

3.4.1.2. 2Fn

Una tabla está en segunda forma normal cuando está en la primera forma normal (1NF) y, además, cada atributo secundario (aquel que no pertenece a la clave principal) depende de la clave principal en su totalidad y no de parte de ella.

Si la clave primaria es simple (formada por un solo atributo), siempre cumplirá la segunda forma normal.

Ejemplo 2FN

Supongamos que tenemos una base de datos de las notas de alumnos de un colegio en distintas asignaturas:

Tabla tbNotas:

idAsignaturaPK (clave primaria)	idAlumnoPK (clave primaria)	nombreAlumno	nota
12	23	Rafael	8
12	24	María	7
13	25	Andrés	5
13	24	María	8

Podemos observar que el atributo "nombreAlumno" depende exclusivamente de idAlumnoPK y no de idAsignaturaPK.

Solución Ejemplo 2FN

Crear una tabla con nombres de alumnos y sacar dichos nombres de la tabla tbNotas. Quedarían las dos tablas siguientes.



Tabla tbNotas:

idAsignaturaPK (clave primaria)	idAlumnoPK (clave primaria)	nota
12	23	8
12	24	7
13	25	5
13	24	8

Tabla tbAlumnos:

idAlumnoPK (clave primaria)	nombreAlumno
23	Rafael
24	María
25	Andrés

3.4.1.3. 3Fn

Una tabla está en tercera forma normal cuando está en la segunda forma normal (2NF) y, además, cada atributo que no forma parte de la clave primaria no depende de ningún otro atributo que no pertenezca a la clave primaria (depende únicamente de la clave primaria).

Ejemplo 3FN:

Supongamos que tenemos la siguiente tabla de empleados:

Tabla tbEmpleados:

idEmpleadoPK (clave primaria)	nombreEmpleado	idDepartamento	Nombre Departamento
1552	Rafael	21	Informática
1232	María	21	Informática
1642	Andrés	22	Contabilidad



Podemos observar que el atributo "nombreDepartamento" depende de "idDepartamento", el cual no forma parte de la clave primaria.

Solución Ejemplo 3FN

Creamos dos tablas:

- tbEmpleados: Contendrá la información de los empleados, incluyendo idDepartamento como clave foránea.
- tbDepartamentos. Recogerá la información de los departamentos.

tbEmpleados:

idEmpleadoPK (clave primaria)	nombreEmpleado	idDepartamento
1552	Rafael	21
1232	María	21
1642	Andrés	22

tbDepartamentos:

idDepartamento (clave primaria)	nombreDepartamento
21	Informática
22	Contabilidad

3.4.1.4. FNBC (forma normal de Boyce-Codd)

Es una versión de la 3FN algo más restrictiva. Una tabla está en la forma normal de Boyce-Codd si está en 3NF y cada dependencia funcional no trivial tiene una clave candidata como determinante.

Por lo tanto, una relación está en forma normal Boyce Codd si cada determinante de la relación es una clave candidata.

Una forma sencilla de comprobar si una relación se encuentra en FNBC consiste en comprobar lo siguiente:

- Comprobar que está en 3FN.
- Si no existen claves candidatas compuestas (con varios atributos) está en FNBC.



- Si existen varias claves candidatas compuestas y estas tienen un elemento común, puede no estar en FNBC.

Solo si, para cada dependencia funcional en la relación, el determinante es una clave candidata, estará en FNBC.

Ejemplo FNBC

Vamos a ver un ejemplo de tabla que está en 3FN y no está en FNBC.

IDDTrabajador	IDDDepartamento	IDDResponsable
50002	Soldadura	Javier
14006	Pintura	Miguel

La única clave candidata es IDDTrabajador (que será por tanto la clave primaria).

Si añadimos la limitación de que el responsable sólo puede serlo de un departamento, este detalle produce una dependencia funcional ya que: IDDResponsable > Departamento.

Por lo tanto, hemos encontrado un determinante (IDDResponsable) que sin embargo no es clave candidata. Por ello, esta tabla no está en FNBC. En este caso la redundancia ocurre por mala selección de clave. La repetición del par [IDDDepartamento + IDDResponsable] es innecesaria y evitable.

Solución Ejemplo FNBC

Para cumplir la FNBC crearemos una tabla de trabajadores-departamento y otra de responsable-departamento. Tendremos dos tablas:

Tabla de trabajadores:

IDDTrabajador	IDDDepartamento
50002	Soldadura
14006	Pintura



Tabla de responsable-departamento:

IDDResponsable	IDDDepartamento
Javier	Soldadura
Miguel	Pintura

3.4.1.5. 4Fn

Para que una tabla esté en 4NF, debe cumplir dos condiciones: primero, debe estar en BCNF (lo que implica que ya está en 1NF, 2NF y 3NF), y segundo, no debe tener ninguna dependencia multivaluada no trivial.

Las dependencias multivaluadas se presentan en relaciones con al menos tres atributos (X, Y, Z) y se da el caso de que Y es multidependiente de X y Z es también multidependiente de X.

Ejemplo 4FN

Tenemos una tabla que nos indica las distintas empresas que distribuyen pollos asados. La tabla tendrá los campos restaurante, tipoSalsa y areaEnvío.

Tabla envioPollo:

restaurante	tipoSalsa	areaEnvío
TelePollo	Sin salsa	Córdoba
TelePollo	Sin salsa	Zaragoza
TelePollo	Salsa canaria	Córdoba
TelePollo	Salsa canaria	Zaragoza
PolloExpress	Sin salsa	Jaén
PolloExpress	Salsa barbacoa	Jaén
PolloFeliz	Salsa canaria	Córdoba
PolloFeliz	Salsa canaria	Jaén
PolloFeliz	Salsa canaria	Zaragoza



Cada fila indica que un restaurante determinado puede entregar pollos con una determinada salsa en un área específica. Como la tabla tiene una clave única y ningún atributo no-clave, no viola ninguna forma normal hasta el BCNF.

Dado que las salsas son independientes de las áreas de envío, hay redundancia en la tabla (por ejemplo, aparece tres veces que PolloFeliz ofrece salsa canaria).

En términos formales, esto se describe como que tipoSalsa está teniendo una dependencia multivalor de "restaurante".



Atención

Suponemos que un restaurante distribuye el mismo tipo de salsas para todas las áreas de envío. Si esto no fuera así (dependiese del área de envío), ya estaría en 4FN.

Solución Ejemplo 4FN

Para satisfacer la 4NF, debemos separar los tipos de salsa de las áreas de envío.

tipoSalsaPorRestaurante:

restaurante	tipoSalsa
TelePollo	Sin salsa
TelePollo	Salsa canaria
PolloExpress	Sin salsa
PolloExpress	Salsa barbacoa
PolloFeliz	Salsa canaria



areaEnvioPorRestaurante:

restaurante	areaEnvio
TelePollo	Córdoba
TelePollo	Zaragoza
PolloExpress	Jaén
PolloFeliz	Córdoba
PolloFeliz	Jaén
PolloFeliz	Zaragoza

3.4.1.6. 5Fn

Una tabla se dice que está en 5NF, si está en 4NF, y cada dependencia de unión (*join*) en ella, es implicada por las claves candidatas.



Recuerda

JOIN (unir, combinar) de SQL permite combinar registros de una o más tablas en una base de datos.

Normalmente, una base de datos que está en 4FN también lo estará en 5FN.

La quinta forma normal (5FN) es también conocida como forma normal de proyección-uni6n (PJ/NF).



Atención

Cuando una relación puede ser reconstruida sin pérdida de información a partir de una combinación de algunas de sus proyecciones se dice que tiene una:

"Dependencia de reunión"

Es un nivel de normalización de bases de datos diseñado para reducir redundancia en las bases de datos relacionales que guardan hechos multivaluados aislando semánticamente relaciones múltiples relacionadas.

Ejemplo 5FN

Tenemos una tabla que nos indica los préstamos de libros realizados en una biblioteca.

prestamoLibros:

Título	Fecha	Socio
El Señor de los Anillos	28/06/2018	0001
Juego de tronos	14/06/2018	0002
El Quijote	17/06/2018	0001
Harry Potter	21/06/2018	0004
El Señor de los Anillos	24/06/2018	0003
Juego de tronos	28/06/2018	0004
El Quijote	21/06/2018	0003

Solución Ejemplo 5FN

Separamos cada par de atributos en una tabla.



tituloFecha:

Título	Fecha
El Señor de los Anillos	28/06/2018
Juego de tronos	14/06/2018
El Quijote	17/06/2018
Harry Potter	21/06/2018
El Señor de los Anillos	24/06/2018
Juego de tronos	28/06/2018
El Quijote	21/06/2018

tituloSocio:

Título	Socio
El Señor de los Anillos	0001
Juego de tronos	0002
El Quijote	0001
Harry Potter	0004
El Señor de los Anillos	0003
Juego de tronos	0004
El Quijote	0003



fechaSocio:

Fecha	Socio
28/06/2018	0001
14/06/2018	0002
17/06/2018	0001
21/06/2018	0004
24/06/2018	0003
28/06/2018	0004
21/06/2018	0003

3.4.2. Desnormalización

Consiste en fundir varias relaciones en una mediante operaciones de tipo JOIN. Esto es útil si se usan juntas con frecuencia, ya que evitamos muchas consultas tipo JOIN.

Ejemplo Desnormalización

Tenemos dos relaciones que se usan juntas con frecuencia:

- Tabla Cliente:

Cliente (codigoCliente, nombre, codigoPostal):

codigoCliente	nombre	codigoPostal
0001	Manuel	50002
0002	Pepe	14420
0003	Paco	50002



Tabla CodigoPostalPueblo:

codigoPostal	codigoPueblo
50002	ZAR
14420	VCO

Solución Desnormalización

Fusionamos las dos en una sola relación.

Cliente (codigoCliente, nombre, codigoPostal, codigoPueblo):

codigoCliente	nombre	codigoPostal	codigoPueblo
0001	Manuel	50002	ZAR
0002	Pepe	14420	VCO
0003	Paco	50002	ZAR



Atención

Hemos introducido redundancias que hay que controlar.

En el ejemplo, un código postal no debe aparecer con dos códigos de pueblo distintos.

Si la primera y tercera tupla tuviesen un valor distinto en el campo codigoPueblo, tendríamos una incoherencia de datos.

3.5. Partición de relaciones

Consiste en dividir algunas relaciones con el objeto de reorganizar la distribución de los casos (partición horizontal) o de los atributos (partición vertical), de manera que una relación incluya atributos o casos a los que se requiera acceso simultáneo con frecuencia.



Ejemplo Partición de relaciones

Empleado (codemp, codEmpresa, nombre, telefono, fechaEvaluacion, resultado).

Supongamos el caso en que se acceda mucho a la tabla para averiguar el nombre y el teléfono, pero no los datos de evaluación. En este caso podríamos realizar una partición.

También, si quisiéramos aislar la información personal de la evaluación (por motivos de seguridad o privacidad), sería conveniente realizar dicha partición.

Solución Ejemplo Partición de relaciones

Al particionar la tabla del ejemplo nos quedaría:

- Empleado (codemp, codEmpresa, nombre, telefono).
- Evaluacion (codemp, fechaEvaluacion, resultado).

Si necesitamos ver la tabla tal y como estaba, tendremos que hacer una consulta de unión.

3.6. Optimización

Consiste en introducir cambios para conseguir un acceso más eficiente.

Ejemplo Optimización

Supongamos que tenemos una tabla con los nombres de los rectores y vicerrectores de cada universidad.

- Universidad (universidad, rector, vicerector).

Supongamos también que cada universidad tiene un rector y de cero a tres vicerrectores.

Para que se encuentre en 2FN tenemos que descomponer la relación en dos:

- Rector (universidad, rector).
- Vicerector (universidad, vicerector).

Siempre que una aplicación necesite la información de una universidad deberá leer varias líneas.



Solución Optimización

Una alternativa que consigue mayor eficiencia (a costa de desnormalizar, ya que no cumple 1FN) es:

- Universidad (universidad, rector, vicerrector1, vicerrector2, vicerrector3).

Hay que tener en cuenta que estamos introduciendo muchos valores nulos en los campos vicerrector1, vicerrector2 y vicerrector3.

La base de datos ocupará más espacio y los valores nulos habrá que tratarlos en la aplicación.

4. Fase 3. Diseño físico



Etapas del diseño físico. Entradas y salidas

El diseño físico parte del resultado del diseño lógico (esquema lógico) y da como resultado una descripción de la implementación de una base de datos en memoria secundaria.

El diseño Físico, define las estructuras de almacenamiento y los métodos que nos permitirán acceder de forma eficiente a los datos.

Esta parte es transparente al usuario.

El diseño físico se adapta al SGBD específico que se va a utilizar. Se expresa haciendo uso de un lenguaje de definición de datos soportado por el SGBD.

El diseño físico no es una etapa aislada, ya que puede provocar modificaciones necesarias en el diseño lógico.



Ejemplo

SQL contiene instrucciones para trabajar como lenguaje de definición de datos. Algunas de estas instrucciones son:

- CREATE DATABASE
- CREATE TABLE
- CREATE SCHEMA
- CREATE VIEW
- CREATE INDEX

Objetivos del diseño físico

El objetivo del diseño físico es describir la implementación de la base de datos en memoria secundaria incluyendo estructuras de almacenamiento y métodos de acceso.



Atención

En el diseño lógico se especifica qué se guarda. En el diseño físico se especifica cómo se guarda, las estructuras de almacenamiento interno y la organización de los archivos.

4.1. Fases de diseño físico

El diseño físico se compone de 4 fases:

- Traducción del esquema lógico global al SGBD específico.
- Diseño de la representación física.
- Diseño de los mecanismos de seguridad.
- Monitorización y optimización del sistema.



4.1.1. Traducción del esquema lógico global al SGBD específico

Esta fase consiste en traducir el esquema lógico global en un esquema que se pueda implementar en el SGBD que hayamos seleccionado.

Se compone de los siguientes pasos:

- Diseño de las tablas (relaciones base) para el SGBD específico.
- Diseño de las reglas de negocio para el SGBD específico.

Diseño de las tablas (relaciones base) para el SGBD específico

Las relaciones base utilizan el lenguaje de definición de datos del SGBD usando la información producida durante el diseño lógico: el esquema lógico global y el diccionario de datos.

El esquema lógico se compone de una serie de relaciones (tablas) y para cada una de ellas tenemos:

- El nombre de la relación.
- La lista de atributos entre paréntesis.
- La clave primaria y las foráneas (si las hubiera).
- Las reglas de integridad de las claves foráneas.

En el diccionario de datos se describen los atributos y, para cada uno de ellos tendremos:

- Su dominio.
- El valor por defecto.
- Si admite nulos.
- Si es derivado y, en caso de serlo, como se calcula su valor.

Diseño de las reglas de negocio para el SGBD específico

Las reglas de negocio de la empresa imponen ciertas restricciones que deben observar las actualizaciones que se realizan sobre las relaciones de la base de datos.

Se pueden implementar estas restricciones de tres formas:

- Mediante mecanismos que proporciona el SGBD específico, los cuales permiten definir las restricciones y vigilar que no se violen.
- Por medio de un disparador (*trigger*).
- Mediante programas de aplicación específicos.



4.1.2. Diseño de la representación física

Uno de los objetivos principales del diseño físico es almacenar los datos de modo eficiente. En ello influyen varios factores que, por norma general, no pueden satisfacerse al mismo tiempo:

- Productividad.
- Tiempo de respuesta.
- Espacio en disco.

El diseñador deberá ir ajustando estos factores para conseguir un equilibrio razonable. Por lo tanto, el diseño físico inicial no será definitivo, sino que habrá que ir monitorizándolo para observar sus respuestas e ir ajustándolo.

El diseñador del esquema físico debe saber cómo interactúan los dispositivos involucrados y como esto afecta a las prestaciones.

Estos dispositivos son:

- Memoria principal.
- CPU.
- Disco (entradas y salidas).
- Red.

4.1.2.1. Pasos para la representación física

Para diseñar la representación física se deben seguir los siguientes pasos:

1. Analizar las transacciones.

Consiste en conocer las consultas y las transacciones con información cualitativa y cuantitativa de ellas.

2. Escoger la organización de ficheros óptima para cada relación.

Estas organizaciones deben documentarse justificando la opción escogida.

3. Escoger los índices secundarios.

Los índices secundarios permiten especificar caminos de acceso adicionales para las relaciones base, pero estos índices conllevan un coste de mantenimiento.

Las razones de su elección deben ser documentadas.



4. Considerar la introducción de redundancias controladas.

Cuando una base de datos normalizada no alcanza las prestaciones deseadas en cuanto a eficiencia, puede ser conveniente desnormalizarla introduciendo redundancias controladas.

Es una opción viable cuando las prestaciones no son las deseadas y la relación tiene muchas consultas y pocas actualizaciones.

Las redundancias pueden ser de varios tipos:

- Datos derivados (calculados a partir de datos).
- Duplicación de datos atributos.
- Hacer JOINS de relaciones (tablas).

5. Estimar la necesidad de espacio en disco.

Esta estimación dependerá del SGBD elegido y del *hardware*.

4.1.3. Diseño de los mecanismos de seguridad

Durante el diseño lógico se especifican los requisitos de seguridad que se implementarán en esta fase. Para ello se seguirán dos pasos:

1. **Diseño de las vistas de usuario** correspondientes a los esquemas lógicos locales.
2. **Diseño de las reglas de acceso.**

El administrador de la base de datos asigna a cada usuario un identificador que tendrá una palabra secreta asociada.

A cada usuario, o grupo de usuarios, se otorgarán permisos para realizar determinadas acciones sobre determinados objetos de la base de datos.

4.1.4. Monitorización y optimización del sistema

Consiste en la puesta en marcha del sistema y su monitorización para observar sus prestaciones. Si estas no fueran satisfactorias, habría que cambiar el esquema.

Este esquema, una vez afinado, tendrá que ir cambiando conforme lo exijan los nuevos requisitos de los usuarios. Los SGBD proporcionan herramientas para monitorizar el sistema mientras está en funcionamiento.



5. Bibliografía

- SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S. *Fundamentos de bases de datos*. 4.ª edición. Madrid: Editorial McGraw-Hill, 2002.
- <http://www3.uji.es/~mmarques/f47/teoria/tema6.pdf>.
- <https://es.slideshare.net/ruthamada/modelo-conceptual-de-la-base-de-datos-360327>.
- <https://es.scribd.com/document/365899939/En-Que-Consiste-El-Modelo-Conceptual-de-Una-Base-de-Datos>.
- <https://en.wikipedia.org>.
- <https://es.wikipedia.org>.
- http://www.unirioja.es/cu/arjaime/Temas/02.Modelo_E_R.pdf.
- <http://bibliotecaprofesional.com/el-modelo-entidad-relacion-en-bases-de-datos/>.
- <https://gsitic.wordpress.com/2018/04/17/biii6-modelizacion-conceptual-el-modelo-entidad-relacion-extendido-e-r-elementos-reglas-de-modelizacion-validacion-y-construccion-de-modelos-de-datos/>.
- https://archivos.csif.es/archivos/andalucia/ensenanza/revistas/csicsif/revista/pdf/Numero_24/ANGEL_COBO_2.pdf.
- <https://es.slideshare.net/ruthamada/modelo-conceptual-de-la-base-de-datos-360327>.
- <https://es.slideshare.net/Viv091/ejemplo-dfd-48917546>.
- ftp://www.dlsi.ua.es/people/jaime/apuntes/aesi_cap4.pdf.
- <https://manuel.cillero.es/doc/metrica-3/>.
- <https://www.smartdraw.com/data-flow-diagram/>.
- <https://www.lucidchart.com/pages/es/qu%C3%A9-es-un-diagrama-de-flujo-de-datos>.
- <https://uvadoc.uva.es/bitstream/10324/12095/5/GUIA%20METODOL%C3%93GICA%20PARA%20LA%20ELABORACI%C3%93N%20DE%20UN%20FLUJOGRAMA.pdf>.
- <https://www.encyclopediaretareas.net/2016/08/reglas-para-hacer-un-diagrama-de-flujo.html>.
- <http://www.areatecnologia.com/diagramas-de-flujo.htm>.



- <https://www.monografias.com/trabajos60/diagrama-flujo-datos/diagrama-flujo-datos2.shtml>.
- <https://vignette.wikia.nocookie.net/bigbangtheory/images/0/0a/Friend2.jpg/revision/latest?cb=20121011222713>.
- https://vignette.wikia.nocookie.net/bigbangtheory/images/6/69/The_Friendship_Algorithm.jpg/revision/latest?cb=20100104100357.
- Piluca Tomás Escobar. Técnico de Hardware, Software, Redes y Programación.
- SILBERSCHATZ, A., KORTH, H., SUDARSHAN, S. *Fundamentos de bases de datos*. 4.ª edición. Madrid: Editorial McGraw-Hill, 2002.
- <https://www.campusmvp.es/recursos/post/Disenando-una-base-de-datos-en-el-modelo-relacional.aspx>.
- <http://www3.uji.es/~mmarques/f47/teoria>.
- <https://es.slideshare.net/SergioRios/unidad-13-analisis-de-requerimientos>.
- <http://elvex.ugr.es/idbis/db/>.
- https://garciagregorio.webcindario.com/sghd/sghd_main.html.
- <https://support.microsoft.com/es-cl/help/283878/description-of-the-database-normalization-basics>.
- <https://es.scribd.com/presentation/38233638/Formas-Normales-FNBC-5FN>.
- <http://es.wikipedia.org>.
- <http://en.wikipedia.org>.
- <https://gsitic.wordpress.com/2018/01/16/biii7-diseno-de-bases-de-datos-la-arquitectura-ansi-sparc-el-modelo-logico-relacional-normalizacion-diseno-logico-diseno-fisico-problemas-de-concurrencia-de-acceso-mecanismos-de-resolucion-de/>.
- <https://www.lucidchart.com/pages/es/tutorial-de-estructura-y-dise%C3%B1o-de-bases-de-datos>.
- http://disi.unal.edu.co/profesores/eleonguz/old/BD_2014_II/presentaciones/S5_normalizacion.pdf.
- https://www.fdi.ucm.es/profesor/fernand/MTIG_/Tema%202%20Dise%C3%B1o.pdf.
- <https://www.campusmvp.es/recursos/post/Disenando-una-base-de-datos-en-el-modelo-relacional.aspx>.
- https://www.ecured.cu/Normalizaci%C3%B3n_de_una_base_de_datos.
- https://www.ecured.cu/Integridad_de_las_Bases_de_Datos#Reglas_de_Integridad.